MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

Authenstickator: a TPM-secured TOTP authenticator app running on a USB stick

MASTER'S THESIS

Master graduate: **Roland BÁLINT**

Supervisor: **Conf. dr. ing. Ciprian Pavel OPRIȘA**

September, 2026

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

DEAN,
Prof.Dr.Eng. Vlad MUREȘAN

HEAD OF DEPARTMENT,
Prof.Dr.Eng. Rodica POTOLEA

Master graduate: **Roland BÁLINT**

Authenstickator: a TPM-secured TOTP authenticator app running on a USB stick

1. **Project proposal:** *Brief description of the master's thesis and the initial data*
2. **Project contents:** *(enumeration of the main parts) Example: Presentation page, Title of chapter 1, Title of chapter 2, . . . , Title of chapter n, Bibliography, Appendices.*
3. **Place of documentation:** *Example:* Technical University of Cluj-Napoca, Computer Science Department
4. **Consultants:**
5. **Date of issuing the proposal:** *Example:* November, 2025
6. **Date of delivery:** *Example:* July 5th, 2026

Master's graduate: (*here comes the signature*) . . . . .

Supervisor: . . . . . .

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

Declarație pe proprie răspundere privind autenticitatea lucrării de disertație

Subsemnatul(a) Roland BÁLINT, legitimat(ă) cu ..... seria ..... nr. ....., CNP ....., autorul lucrării Authenstickator: a TPM-secured TOTP authenticator app running on a USB stick, elaborată în vederea susținerii examenului de finalizare a studiilor de disertație la Facultatea de Automatică și Calculatoare, Programul de studiu ....., din cadrul Universității Tehnice din Cluj-Napoca, sesiunea .....(iulie/septembrie) .....a anului universitar 2024-2025, declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de disertație*.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine.

Data

.....

Roland BÁLINT

.....
Signature

Contents

Chapter 1

Introduction

Chapter 2

Objectives of the Research

The objective if to provide a secure, modular and portable architecture for a TOTP authenticator on a USB stick, and a software solution implementing it.

Authenstickator replaces a regular authenticator app like Microsoft's Authenticator or Google Authenticator. For these, you need a mobile phone. If you don't have a dedicated work phone, you might need to tie your personal device to the TOTP verification of a work account. In case you lose your phone, your company might require to wipe your data. Furthermore, mixing personal belongings with work is not a good practice. A USB device is cheap, so it is not an overhead to have a USB device solely for authentication purposes.

While professional solutions like YubiKeys exist, they are quite expesive due to their dedicated hardware and branding. While they are far more secure than a simple USB stick, for two-factor authentication Authenstickator is more than enough.

Chapter 3

Bibliographic Research

This chapter will go through the main ideas, principles, algorithms, and notions used throughout the implementation of Authenstickator. They will be presented as found in literature, with references to bibliographic sources, hence the chapter's title. While the sections don't necessarily come in a given order, I tried to structure them from the main ideas to the more detailed aspects. Each section can be read in isolation, so feel free to jump to the one of your interest.

3.1 Existing Solutions

In this section I will present software solutions (sometimes backed by hardware) which solve a similar problem to what Authenstickator aims to solve. They will be presented in comparison with Authenstickator, so you will see both arguments for and against my solution. Frankly, I found no other project doing exactly what I do, so none of the presented solutions will be a one-to-one match to my project. Also, when comparing, keep in mind that Authenstickator is developed by a single person, in a limited amount of time, and serves more as a proof of concept rather than a polished end product.

Without further ado, lets dive deep into the "opponents" of Authenstickator.

3.1.1 YubiKey

The YubiKey is a product if the company called yubico. It has gained rising popularity over the last few years, both for personal and professional use. It comes in various shapes and forms, but mostly resembles a USB stick. However, these are custom-built hardware designed specifically to support different protocols which need a security key.

Citing from their website [1], "The YubiKey works with hundreds of enterprise, developer and consumer applications, out-of-the-box and with no client software. Combined with leading password managers, social login and enterprise single sign on systems the YubiKey enables secure access to thousands of online services." Their products also support OTP: "OTP supports protocols where a single use code is entered to provide authentication. These protocols tend to be older and more widely supported in legacy applications. The YubiKey communicates via the HID keyboard interface, sending output as a series of keystrokes. This means OTP protocols can work across all OSs and environments that support USB keyboards, as well as with any app that can accept keyboard input."

Their most recommended product is YubiKey 5, and they call it the world's number one multiprotocol security key. From these series, as of August 21st, 2026, the cheapest for 70.18 EUR. Other models from this series go even more expensive, with the biggest price being 102.85 EUR. The models differ in capabilities, but also physical size, the smaller ones costing generally more.

Yubico is a generally trusted company with a good reputation. It is based in Stockholm, Sweden, and was founded in 2007 by Stina Ehrensvard. According to Tracxn[1], it is ranked 1st among its competitors. They have around 300 employees and have a total of 30M USD funding. However, the software that runs in YubiKey is closed-source. They did have some security issues in the past[2], including:

- **ROCA vulnerability - October 2017**. A vulnerability which affected many security keys which implemented RSA. It allowed an attacker to reconstruct the private key by using the public key. Yubico gave free replacements with newer series devices.
- **OTP password protection - January 2018** This affected a specific model, the YubiKey NEO. The password protection for the OTP functionality could be bypassed. Yubico offered a firmware update and free replacements.
- **Cloning vulnerability - September 2024** It allowed a person to clone a YubiKey if they gave physical access. It was remediated through a firmware update. This required advanced cryptographic and technical knowledge: they looked at the electromagnetic emissions of the keys. [2]

In comparison with Authenstickator, YubiKeys are much more secure, since they are specialized hardware. It is backed by a stable company with a team of software and hardware engineers. It went through various iterations, and leads in the domain of security keys. They are a solution for various problems, OTP being just one tiny functionality of theirs. They are widely used, also by big corporations.

In defense of my project, Authenstickator also has some benefits compared to a YubiKey. First, the price of Authenstickator is bond to the price of the USB key it is used on, which is minuscule compared to 70 EUR. Most of us have a spare USB stick laying around without a purpose, and with Authenstickator, we can give life to a potential electronic waste. Second, the source code of Authenstickator is fully open source, so anyone can audit it or reuse it (respecting its GPL-3.0 license). Authenstickator is an academic project, while YubiKeys are professional products.

3.1.2 Virtual FIDO

Citing from the GitHub repository of Virtual FIDO [3]: " Virtual FIDO is a virtual USB device that implements the FIDO2/U2F protocol (like a YubiKey) to support 2FA and WebAuthN. Virtual FIDO creates a USB/IP server over local TCP to attach a virtual USB device. This USB device then emulates the USB/CTAP protocols to provide U2F/FIDO services to the host computer. "

So, this is a fully-software solution for FIDO2 authentication, which is a different protocol than TOTP. However, both FIDO2 (Fast IDentity Online) and TOTP are used

[1] `https://tracxn.com/d/companies/yubico/__fgcOVhxhBthY3y9De52OtlO21ENbj9VtVBf2Ho1uoOI#about-the-company`

[2] `https://en.wikipedia.org/wiki/YubiKey#Security_issues`

[3] `https://github.com/bulwarkid/virtual-fido`

for Multi-Factor Authentication (MFA). While TOTP strengthens traditional authentication, FIDO2 eliminates the need for passwords completely.

Looking over the fact that it implements a different protocol, Virtual FIDO is also a solution for MFA. While at first it might sound great that we have this software running on the host machine without an external device, on a second thought you might realize that it completely kills the MFA aspect. It is rather a bypass for FIDO, and should never be used. Possible use-cases could be development and testing, but other than that, the idea is pretty wrong.

So, Authenstickator wins over Virtual FIDO, as Authenstickator tries to provide a security solution, not to bypass one. It serves as a good example that throughout the design of the project, I had to keep in mind that we absolutely want the software (and the secrets) to run/live on the USB, not on the host machine.

3.1.3 USB Raptor

USB Raptor is a free and open-source project designed for Windows, which transforms a USB flash drive into a computer lock and unlock key. Citing from their SourceForge page:[4] " USB Raptor can lock the system once a specific USB drive is removed from the computer and unlock when the drive is plugged in again to any USB port. The utility checks constantly the USB drives for the presence of a specific unlock file with encrypted content. If this specific file is found the computer stays unlocked otherwise the computer locks. To release the system lock user must plug the USB with the file in any USB port. Alternative the user can enable (or disable) two additional ways to unlock the system such is network messaging or password. "

Again, this application is not quite what Authenstickator aims to be, but it also tries to convert a USB stick into a security device. It is not quite a security key, although some websites name it so. The application runs in the background in the system tray using minimal resources. Its behavior is highly customizable, from sound effects to lock delays. It provides backup alternatives if you get locked out, like a RUID file.

After some light research, I came across a Reddit post[5] which askes how to bypass USB Raptor. The poster wants to do so since USB Raptor stopped working on his/her machine. The correct password was not recognized by the software, leading to system lockout. Multiple users report having the same issue, one saying that his/her system had to be wiped. In the same SourceForge page which provides the application, in the Wiki section's comment area I found multiple users asking for unlock keys for exchange of a donation, reminding me of ransomware. The posted source code is also quite outdated, with releases after the last update, so the project cannot be considered fully open source.

It might be paranoia from my side, but based on what findings I described earlier, I would not consider USB Raptor trusted software. While the idea is interesting, it can lead to unpleasant side effects. USB Raptor finds a different way to turn a USB stick into a security device than Authenstickator. I consider my solution to be safer.

[4] `https://sourceforge.net/p/usbraptor/wiki/Home/`

[5] `https://www.reddit.com/r/ComputerSecurity/comments/18ul3ex/how_do_i_bypass_usb_raptor_my_computer_refuses_to/`

3.2 Two-Factor Authentication

"Two-factor authentication is a subset of multi-factor authentication (MFA). It requires two authentication factors to allow access to a service or system. However, two factors from the same category don't constitute two-factor authentication."[3] It is a security concept aimed to reduce the volume of cyberattacks against authentication.

So what are these authentication factors? Authentication is about ensuring the person who says "I am John Doe" is actually John Doe. It all comes down to how he proves his identity. There are several authentication factor types, but the most common are:

1. **Knowledge factor.** Answers the question: what does only the user know? It shall be an information that only one specific person has available. Common examples are passwords, personal identification numbers (PINs), passcodes.

2. **Possession factor.** Answers the question: what does only the user have? Physical possessions, like a driver's license, an identification card, or a mobile device all go into this category. A common scenario is an authenticator application on a smartphone, or push notifications/ SMSes sent to it.

3. **Location factor.** The location of the user at the moment of authentication is used as a restrictive criterion. This is mostly suitable for organizations/ corporations which are active in fixed locations.

4. **Time factor.** Similar to the location factor, but the restrictive criterion being the time of login. Access requests outside a permitted time range are declined and/or reported.

Two-factor authentication is not a new concept. It has been around for quite some time. It has also received criticism, since many doubt their effectiveness. Bruce Schneier, an American cryptographer, computer security professional and adjunct lecturer at the Harvard Kennedy School writes in an article dating back to 2005[4]: "Two-factor authentication isn't our savior. It won't defend against phishing. It's not going to prevent identity theft. It's not going to secure online accounts from fraudulent transactions. It solves the security problems we had 10 years ago, not the security problems we have today." He mentions man-in-the-middle attacks and Trojan malware as two examples which completely bypass two-factor authentication.

Indeed, no security measure is medicine against all cyberattacks. However, there are studies which show a significant increase in security when MFA is enabled compared to when it's not. A study from 2023 conducted by Microsoft employees investigated the effectiveness of MFA in protecting commercial accounts from unauthorized access.[5] They underline that is hard to come with exact numbers in this regard, since compromised users often do not report being compromised. Their article states: "We obtained a sample of 128,000 accounts that had their passwords leaked between April and September of 2022. Users were immediately notified. We retroactively studied those accounts for 30 days prior to the discovery of the credential leak. Reviewing the accounts manually, we found 7,861 accounts that had MFA and for which we could confirm that attackers used the passwords to try to obtain access to protected resources. For those accounts, we found that MFA prevented 98.6% of the attacks."

The same paper mentions a study by Google made in 2019, which found that "challenges and MFA prevented 100% of automated attacks, 96% of bulk phishing attacks, and 76% of targeted attacks." So, while there are ways to circumvent MFA (and implicitly, 2FA), it provides significant security gains especially for big corporations which handle sensitive data. These accounts are often more targeted than personal accounts, and while sophisticated attack methods are out of scope, a big wave of automated attacks and cheap tricks can be solved by two-factor authentication.

3.3 One-time Password Algorithms

"A one-time password (OTP) is a temporary, auto-generated code used to authenticate a user for a single login session or transaction."[6] It is a user-friendly string of characters or numbers, i.e., it can be easily entered manually when prompted to do so. While a traditional, static password is permanent, one-time passwords expire in a short period of time to combat brute-force attacks. They are widely used as a second factor: if it is sent to or generated by a separate device than the one the application runs on, it becomes the second (possession) factor along the regular password (the knowledge factor).

3.3.1 HMAC-Based One-Time Password Algorithm

The HMAC-Based One-Time Password Algorithm, or HOTP for short, is an open algorithm proposed by D. M'Raihi et al.[6], backed by The Internet Society. It was published in 2005. It focuses on defining an algorithm which would have minimal hardware requirements, but be secure. Before its publication, it seems like there were no successful standardization attempts regarding OTP. They mention Java smart cards, USB dongles, and GSM SIM cards, but in today's world, the device is mostly a smartphone.

"The HOTP algorithm is based on an increasing counter value and a static symmetric key known only to the token and the validation service. In order to create the HOTP value, we will use the HMAC-SHA-1 algorithm, as defined in RFC 2104. As the output of the HMAC-SHA-1 calculation is 160 bits, we must truncate this value to something that can be easily entered by a user." They mention the following formula:

$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA-1}(K, C)) \tag{3.1}$$

Where *Truncate* represents the function that converts an HMAC-SHA-1 value into an HOTP value, K is the Key and C is the Counter.

Put in simple words, there is a counter which increases with each usage, and a shared secret. By shared, I mean that it is known both by the system which wants to authenticate and the system which validates the authentication, but no one else. The secret is generated by the authentication system and sent to the user once, at a provisioning step. How this is done is out of scope of the specification. The counter and the secret are combined by a hashing function, and converted to a user-friendly format. This HOTP value can be calculated both by the token and the validation service, since their parameters are known by both.

One drawback of this algorithm (considering the problem of sharing the secret solved) is that the counters have to be kept in sync. The paper treats this issue in

[6] `https://www.okta.com/blog/identity-security/what-is-a-one-time-password-otp/`

its Resynchronization of the Counter section. They recommend setting a look-ahead parameter, basically defining a window for the accepted counters server-side. This allows some drifting of the counter. Resynchronization is also an option, but it is "even more difficult than guessing a single HOTP value". Aside of this, like with any other security-related algorithms, parameters have to be chosen with care, such that the algorithm balances between usability and security.

3.3.2 Time-Based One-Time Password Algorithm

Time-Based One-Time Password Algorithm (TOTP) is a derivation of the HMAC-Based One-Time Password Algorithm described in Section 3.3.1. The related RFC is published by the Internet Engineering Task Force in May 2011[7], one of the authors being D. M'Raihi itself (also found in the editors of [6]). As they state, "The HOTP algorithm specifies an event-based OTP algorithm, where the moving factor is an event counter. The present work bases the moving factor on a time value. A time-based variant of the OTP algorithm provides short-lived OTP values, which are desirable for enhanced security."

In other words, instead of the Counter C in Equation 3.1, TOTP uses an integer T which represents the number of time steps between the initial counter time and the current Unix time. The time step X is a system parameter, which represents the time step in seconds, i.e., for how many seconds is a password valid; the recommended default is 30 seconds. The initial counter time $T0$ is the time from which we start counting the steps; the recommended default is 0, so simply the current Unix time is used. For example, if $T0 = 0$ and $X = 30$, $T = 1$ if the current Unix time is 59 seconds, and $T = 2$ if the current Unix time is 60 seconds.

$$TOTP = \text{HOTP}(K, T) \tag{3.2}$$

$$T = \frac{(CurrentUnixTime - T0)}{X} \tag{3.3}$$

Similarly to HOTP, a time window for tolerance can be defined to deal with clock shifts. Naturally, a requirement for TOTP is the ability of the devices to get the current Unix time. With most digital devices, this is not a problem, but clock synchronization is not a trivial problem. "If the time step is 30 seconds as recommended, and the validator is set to only accept two time steps backward, then the maximum elapsed time drift would be around 89 seconds, i.e., 29 seconds in the calculated time step and 60 seconds for two backward time steps." Usually this is more than enough to tolerate clock shifts.

The TOTP algorithm generates the 6-digit codes you are probably familiar with. The users usually download a mobile authenticator application. Figure 3.1 shows three popular authenticator applications, namely, from left to right: Proton Authenticator, Microsoft Authenticator, Google Authenticator. The provisioning step is done by showing a QR code which the user can scan with the application, or the secret may be added manually. After provisioning, the application stores the secret.

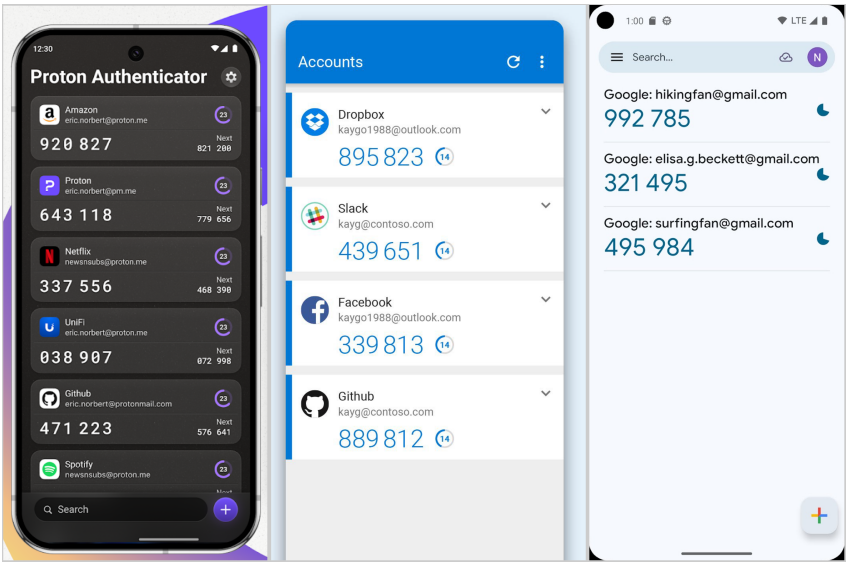


Figure 3.1: Authenticator apps. Source: Google Play.

3.4 Encryption and AES

"Encryption is the process of transforming readable plain text into unreadable ciphertext to mask sensitive information from unauthorized users."[7]. It is a two-way operation: a plaintext can be encrypted into ciphertext, and a ciphertext can be decrypted into plaintext. Encryption and decryption happens with the help of keys (secrets). If the same key is used both for encryption and decryption, the algorithm is a symmetric encryption algorithm. Asymmetric encryption uses two keys: a public key for encryption and a private key for decryption.

Both asymmetric and symmetric encryption algorithms have strenghts and weaknesses. Asymmetric encryption is slower, but more secure since eliminates the need of key exchange. Symmetric is faster, but the key has to be exchanged. Usually (ex: for TLS), asymmetric encryption is used to exchange a symmetric key, and from then on, symmetrix encryption is used (for a given time period, after which a new key is exchanged, the old key becoming expired). Encryption is a must-have for sending sensitive information through untrusted media, like the internet, but also essential to make sure data at rest is protected (file encryption, disk encryption, database encryption).

The Rijndael Cipher is the winner of the AES competition[8]. It is the golden standard of symmetric encryption algorithms. It transforms a plaintext of a given size into a ciphertext of the same size. The plaintext is encoded in hexadecimal, and split into 128-bit blocks. One block can be represented as a 4 by 4 matrix, where each cell contains two hexadecimal characters. Since each hexadecimal character can represent 2 bytes, one cell is 4 bytes, one row is 16 bytes, and the whole matrix is 128 bytes (4 rows).

The AES algorithm repeatedly applies four types of transformations as depicted in Figure 3.2. The figures in this section are from the Rijandael (AES) Animation made by Forma Estudio[9]. This animation explains the algorithm much better than my text, but here is my attempt. The transformations are:

1. **SubBytes**. Uses an S-BOX, i.e., a lookup table. Each cell is substituted (changed) with a value from the S-BOX. The value is chosen by taking the first byte from the

[7] `https://www.ibm.com/think/topics/encryption`

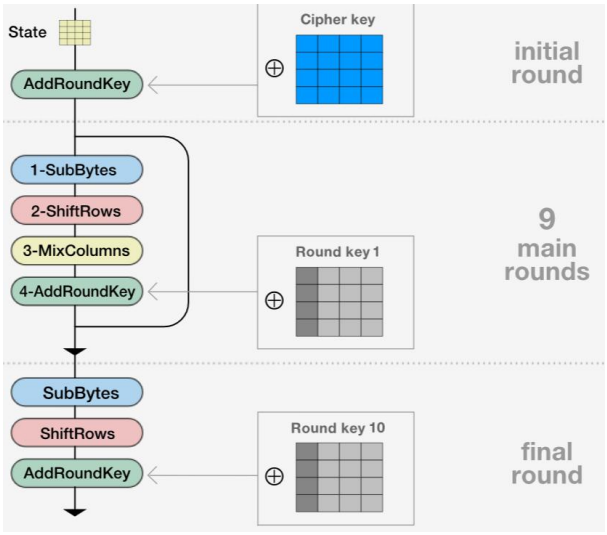


Figure 3.2: AES encryption steps. Source: Forma Estudio.[9].

cell to select the row, and taking the second byte to select the column.

2. **ShiftRows**. Each row is rotated to the left with its index. The first row stays as-is, the second is rotated 1, the third is rotated 2, and the fourth is rotated 3.

3. **MixColumns**. Each column is Galois-multiplied by a specific matrix. Galois-multiplication is a special type of multiplication, which gives results in the same set as its inputs. So this is yet another step to mix bytes up deterministically.

4. **AddRoundKey**. The round key is XOR-ed with the current matrix (explained later).

The **AddRoundKey** step mentions a round key. Round key comes from the Key Scheduling part of the Rijandael algorithm. The cipher key (input of the algorithm) is expanded into eleven partial keys: one for the initial round, nine for the main rounds, and one for the final round. We can visualize the cipher key as a matrix yet again, and the round keys will be generated to its right, column by column (see: Figure **??**). Starting with the cipher key, up until all the columns are generated, the steps below are performed. Note that, the steps marked with * are performed only if the column's position is a multiple of 4. For the rest of the columns, only XOR applies.

1. **RotWord***. the column is rotated once, in upside direction.

2. **SubBytes*** Same as for enctyption, using the same S-BOX.

3. **XOR** The column is XOR-ed with the column 4 positions to the left.

4. **Add*** The column is summed with a column from a specific constant matrix (Rcon)

Now the amazing part is that encryption can be done the same way, but by reversing everything. Instead of adding a matrix, we subtract it. Instead of shifting to one direction, we shift to the opposite direction. For substitution, we do the inverse substitution. And Galois-multiplication has its inverse operation as well. While some of these operations look complicated graphically, computers are really good at performing them, making encryption and decryption fast. For its strong security, open-source nature and performance, AES is used in almost all cases when symmetric encryption is needed.

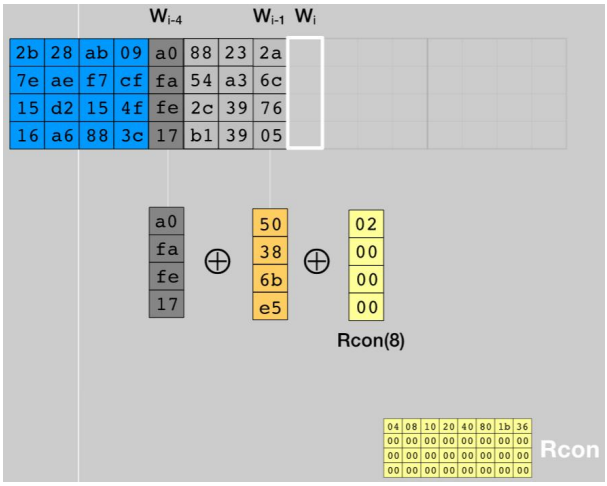


Figure 3.3: AES Key Schedule. Source: Forma Estudio.[9].

3.5 Password-Based Key Derivation. PBKDF2

"In cryptography, a key derivation function (KDF) is a cryptographic algorithm that derives one or more secret keys from a secret value such as a master key, a password, or a passphrase using a pseudorandom function (which typically uses a cryptographic hash function or block cipher)"[8]. They are often used when keys need to be stretched or need to have a certain format. A typical use case is applying password-based key derivation on a key before using it for AES. If the key needs to be of a certain length, depending on the context, just padding it might break the security of the following algorithms, since it is predictable and not random.

Key derivation functions can not only enable random user input to be stretched to the required size, but it usually makes them harder to break as well. These functions are intentionally slow, so that brute-force attacks and dictionary attacks are harder to be performed if inputs are being guessed. If the key derivation is deliberately slow enough, it can be considered a hash function. Such algorithms are, for example, crypt, bcrypt and scrypt.

The Password-Based Key Derivation Functions are examples of key derivation functions. I used the word in plural since there are to major versions of it, PBKDF1 and PBKDF2. "PBKDF1 applies a hash function, which shall be MD2, MD5 or SHA-1, to derive keys. The length of the derived key is bounded by the length of the hash function output, which is 16 octets for MD2 and MD5 and 20 octets for SHA-1. (...) PBKDF2 applies a pseudorandom function to derive keys. The length of the derived key is essentially unbounded."[10]

I will focus on PBKDF2, as even the official publication describing them recommends it over PBKDF1. The PBKDF2 algorithm has the following parameters:

- P. A password, as an octet string (input). This is the password from which we want to derive a key. Usually user input, and we cannot rely on it being of a specific length or "strength".

- S. A salt, as an octet string (input). Just as usually, the salt has the purpose to bring randomness to the output. Even if the same password is provided, given

[8] `https://en.wikipedia.org/wiki/Key_derivation_function`

that the salt is different, the hash will be different. This is a mechanism against brute-force and dictionary attacks.

- c. An iteration count, as a positive integer (input). It serves the purpose of increasing the cost of producing keys from a password. This way it also increases the difficulty of attack, making brute-forcing or building a dictionary take more time. The recommended minimum number of iterations is 1000.
- $dkLen$. The desired length of the derived key (input). This depends on what we want the key to use for later. But as principle, it can be different numbers, so the algorithm is capable of deriving different length keys.
- PRF The underlying pseudorandom function. Depending on which pseudorandom function is used, it might consume different length octets at a time. This length is refered to as $hLen$

First, the number of $hLen$-octet blocks of the derived key is calculated. We simply take the total desired length and divide with the length taken at once, and ceil the result.

$$l = \left\lceil \frac{dkLen}{hLen} \right\rceil \tag{3.4}$$

The number of leftover octets (for the last block) will be denoted with r.

$$r = dkLen - (l - 1) \times hLen \tag{3.5}$$

Each block of the derived key (T_i) will come from the function F.

$$T_i = F(P, S, c, i) \quad \text{for } i = 1, 2, \ldots, l \tag{3.6}$$

And the function F is the XOR of the first c iterations.

$$F(P, S, c, i) = U_1 \oplus U_2 \oplus \cdots \oplus U_c \tag{3.7}$$

Where each iteration U_i comes from applying PRF on the password and different salts. For the first iteration, the salt is S concatenated with an encoded version of i (the block index). For the rest of the blocks, the salt is the result of the previous iteration.

$$\begin{aligned} U_1 &= \mathrm{PRF}(P, S \parallel \mathrm{INT}(i)) \\ U_2 &= \mathrm{PRF}(P, U_1) \\ &\vdots \\ U_c &= \mathrm{PRF}(P, U_{c-1}) \end{aligned} \tag{3.8}$$

After each block is calculated, they are concatenated. From the last block we only take the first r octets. The result is the derived key, having the desired length.

$$DK = T_1 \parallel T_2 \parallel \cdots \parallel T_l\langle 0 \ldots r - 1 \rangle \tag{3.9}$$

3.6 Hashing and Argon2

"Hashing is a process that converts any input to a fixed-length, seemingly random output."[9]. For a function to be called hashing function, it has to fulfill certain criteria, aside of giving the same length output for any length input. First of all, it has to be a one-way function. Unlike for encryption, where we want the ciphertext to be decryptable, in case of hashing, it shall be impossible to convert the output of a hash function back to the input which it came from. Hashes are compared between each other, or used as they are, but never reversed.

Then, it has to have good collision resistance. Since we convert to a fixed size string, multiple inputs will give the same output, inevitably. It is like parking infinite cars, in seemingly random order, into finite parking lots. At one point, parking lots will fill up, and two cars will have the same lot. This means that, if used for password storage and authentication, there is a mathematical chance that two passwords give the same hash, even with salting. This would mean logging in with an incorrect password. The catch is that for hash functions with good collision resistance, the chances are terribly low. For example, for the MD5 algorithm, which was broken in 2004 (it was shown that it is not collision-resistant), the probability to have two passwords producing the same hash is $\frac{1.47}{10^{29}} = 0.00000000000000000000000000147\%$.[10].

The third criterion for a hash function to be considered usable is to be deterministic, not random. That is why, in the last paragraph's analogy, I said "seemingly random". The same input must give the same output all the time. Otherwise, it would just be a random number generator, without the current practical use cases.

Hash functions are used for various purposes, out of which the most common are:

- **Password storage.** Databases are often leaked, so storing the hash of a password instead clear text is the 101 of cybersecurity. In this case, it is extremely important to use a strong hashing algorithm, with proven collision resistance.

- **Fast look-up data structures.** Hash maps, for example, are often used where data has to be retrieved in `O(1)` time complexity. If we want to look for something, we compute its hash, which is the address it is supposed to be at. This way, we can instantly look up data. Collision resistance in this use-case is important for maintaining the speed advantage, not for security.

- **File signatures.** A hash function can take the contents of a whole file and convert it into a much shorter hash. This way, we get a unique value, a signature for each file (again, given that no collisions happen). We can easily check if a file changed, by looking at the previous and current signature. File signatures are also used for static malware analysis: we can take the hash of a program, feed it to a database like `https://www.virustotal.com`, and check if it is a reported malware.

Argon2[11] is a password hashing algorithm, the winner of the Password Hashing Competition. "PHC ran from 2013 to 2015 as an open competition—the same kind of process as NIST's AES and SHA-3 competitions, and the most effective way to develop a crypto standard. We received 24 candidates, including many excellent designs, and

[9] `https://www.bitdefender.com/en-us/business/infozone/what-is-hashing`
[10] `https://infosecscout.com/two-passwords-same-hash/`

selected one winner, Argon2, an algorithm designed by Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich from University of Luxembourg."[11].

The algorithm aims to maximize the cost of password cracking on Application-Specific Integrated Circuits (ASICs). An ASIC is An "integrated circuit designed for a specific customer, application, or market (...) interconnected, and simulated to provide the desired system functions and level of performance." [12]. In this context, it refers to hardware specialized for password cracking. The advanced attackers who use such software often exploit parallelization (on GPUs, FPGAs, with more cores), they compute blocks ahead, they juggle with time-memory tradeoff ("the adversary can trade memory for time and do his job on fast hardware with low memory."[11]).

Argon2 takes all these aspects into account, and comes with a solution in two flavors: Argon2d and Argon2i. Argon2d is faster and uses data-depending memory access, suitable for cryptocurrencies, for example. Argon2i is slower and uses data-independent memory access, which is preferred for password hashing. Data-dependent indexing takes into account the password when choosing the memory index to write to, this way making it impossible to prefetch and precompute missing data. Some software, like Bitwarden, use Argon2id, which uses "a combination of data-depending and data-independent memory accesses, which gives it some of Argon2i's resistance to side-channel cache timing attacks and much of Argon2d's resistance to GPU cracking attacks."[13]

The mathematics behind Argon2 are out of scope for this document, but simply said, the algorithm consists of three steps. First, memory initialization fills a defined memory size with pseudorandom values derived from the password, salt, and other parameters. Then, the memory's contents are repeatedly combined with the password and the salt. After all the iterations, the used memory is compressed into a fixed-size output (the hash).[14]. All this makes Argon2 an outstanding choice for password hashing, proven to be up-to-date with today's attackers.

3.7 Full Disk Encryption and LUKS

Full disk encryption, as its name suggests, refers to encrypting every byte of a physical disk. This mechanism is especially powerful since it provides a layer of security for the whole contents of a machine. It also works for removable media. With full disk encryption, an attacker's physical access does no longer mean compromised data. With powerful encryption, brute-forcing decryption of a full disk takes so much time that it is virtually impossible. "LUKS (Linux Unified Key Setup) is the standard disk encryption format for Linux, built on top of the dm-crypt kernel module. It is the encryption layer used by default when you select "encrypt disk" during installation on Fedora, Ubuntu, Debian, and most other major distributions."[15].

It uses PBKDF2 to enhance weak user passwords and has a two level key hierarchy.[12] This means that a master password is generated, which is not shared with the user, and the user's password is used to encrypt this master password, which is then stored in a slot. There are several such slots (key material sections) in the LUKS header, meaning

[11] https://www.password-hashing.net/

[12] https://www.analog.com/media/en/analog-dialogue/volume-24/number-3/articles/volume 24-number3.pdf#page=12

[13] https://bitwarden.com/help/kdf-algorithms/

[14] https://jumpcloud.com/it-index/what-is-argon2

[15] https://secure-os.org/articles/luks-encryption/

that multiple passwords can be used to decrypt the disk. As a result, one can have a recovery key along the usual user password.

The LUKS header is the first few bytes of the disk which contains metadata needed for decryption (the algorithm, the KDF etc.). It is a good idea to have a periodic backup of the LUKS header, since if this part of the memory is damaged, the whole disk becomes inaccessible. It is a single point of failure, but there is no free meal.

LUKS2, the newer variant of LUKS, is a good option for encrypting disks and USB devices. As with other cryptographic algorithms, there have been vulnerabilities in their implementation: in 2016, you could gain access by simply holding the enter button.[16]. But it is a good standard, and its open-source implementation is patched regularly.

3.8 TPM

"A Trusted Platform Module, also known as a TPM, is a cryptographic coprocessor that is present on most commercial PCs and servers" [13]. This chip provides hardware-backed cryptographic primitives upon which software applications can build on. Usually, when TPM is mentioned, it refers to the chip, which can come from different manufacturers, but must implement the specification defined by the Trusted Computing Group.

There are two main TPM specifications, one of them being TPM 1.2 (ISO/IEC 11889:2009) and the newer one being TPM 2.0 (ISO/IEC 11889:2015).[17]. TPM 2.0 is not backward compatible with TPM 1.2. While both specify similar features, TPM 2.0 is broader and is being pushed over its predecessor. Since 2016, Microsoft has required all new PCs shipping with Windows 10 to include TPM 2.0 support. When they released Windows 11 in 2021, the requirements contained TPM 2.0 support. This caused major backlash, since many older PCs did not (and still do not) have this. It is hard to find up-to-date exact statistics related to TPM 2.0 adoption rate, probably the least speculative number comes from Windows 11 adoption, as published by Microsoft (given that we trust them). In March the 3rd, 2026, they stated that "Windows 11 Dominates Desktop Share in 2026: 73% of Windows PCs".[18]. We can assume that most of these machines have TPM 2.0 (with some technical knowledge, or endorsement from Microsoft, this requirement can be bypassed).

A TPM is often refered to as a cryptographic coprocessor, a hardware security module (HSM) or a hardware root of trust (HToT).[14] Its cryptographic functionalities include Random Number Generation (RNG), key generation, provides hashing functions and Hash Based Message Authentication Code (HMAC), as well as implementations for SHA-1, SHA-256, SHA-384, RSA, ECC, AES, SM4 and XOR. It also comes with volatile and non-volatile memory. Volatile memory is used for authorization sessions and storing active keys. Non-volatile memory has modules such as the Platform Configuration Registers (PCRs) and a limited Non-Volatile Random Access Memory (NVRAM).

The primary scope of TPM is to verify integrity during the boot process. Its PCRs contain values which can be used "to cryptographically record (measure) software state: both the software running on a platform and configuration data used by that

[16] https://hmarco.org/bugs/CVE-2016-4484/CVE-2016-4484_cryptsetup_initrd_shell.html

[17] https://en.wikipedia.org/wiki/Trusted_Platform_Module

[18] https://windowsforum.com/windows-news.4/windows-11-dominates-desktop-share-in-2026-73-of-windows-pcs.403819/?spm=a2700.accio_bizSeo.0.0.180331dah3UXgl

software."[13]. The state is hashed and the hash is used when booting to verify is the system was altered or not.

The NVRAM is used to store two types of data: data structures defined by the TPM architecture and unstructured data defined by a user. An example use case mentioned by [13] is storing a secret. This memory is very limited in size, so it is not for storing large files. Instead, the encryption key for a large file could be stored.

All in all, TPM 2.0 provides various hardware-backed cryptographic primitives, which could be used by the OS, but also by software running on a machine. The TPM in itself is not a solution to all security problems neither. Probably the biggest criticism of it came from the VeraCrypt developers, stating that "The only thing that TPM is almost guaranteed to provide is a false sense of security."[19]. Indeed, if the device is stolen, it is just a matter of time, technical skills and available resources before it is bypassed. However, using its cryptographic primitives instead of software solutions is undoubtably stronger, with one catch - we have to trust the TPM manufacturer and vendor.

[19] `https://veracrypt.io/en/FAQ.html`

Chapter 4

Project Presentation

4.1 Proposed Framework

The goal of this Masters' thesis is to provide a generic framework for a TOTP application on a USB stick, as well as an implementation for it. In this section, we will focus on the framework, i.e., the ideas and principles, rather than the technology used. The presented framework could work on other devices as well, it is a generic flow of actions meant to be abstract. At the same time, the framework's main goal is security, so it has to take into account all kinds of attacks and countermeasures for them. With all that in mind, the framework focuses on minimizing the possibility of exploits, errors, deadlocks.

Simply put, a TOTP application generates TOTP codes (numbers) based on TOTP secrets (Base32 codes). The application which requires TOTP (example: Gmail, Facebook, etc.) displays a QR code or an equivalent secret. The user scans the QR code or introduces the secret manually into the TOTP application, which usually runs on a personal mobile device. From that point on, the TOTP application stores that secret in a way or other, and keeps generating TOTP codes.

The framework aims to replace the mobile device with a USB stick. This goal shall be achieved by fulfilling the following requirements:

1. **Generate TOTP codes**. This is the main functional requirement of the framework. TOTP code generation is a standard algorithm and requires only the current time and the TOTP secret.

2. **Store the secrets safely**. TOTP secrets are sensitive information. They shall be stored in a way which is secure and resistant to possible cyber-attacks.

3. **Keep the 2FA aspect**: The framework treats the device on which the app runs as the possession factor of Two-Factor Authentication. Breaking this aspect would make the framework useless.

Figure 4.1 shows the two main flows of the proposed architecture. It proposes the combination of three secrets: a password for encrypting the USB stick (**disk encryption password**), a password for unlocking the application and encrypting the secrets (**user password**), and optionally a TPM-provided secret which is combined with the user password for extra security (**TPM secret**). Each of these will be detailed in the following sections. The following sections will go in depth on each aspect.

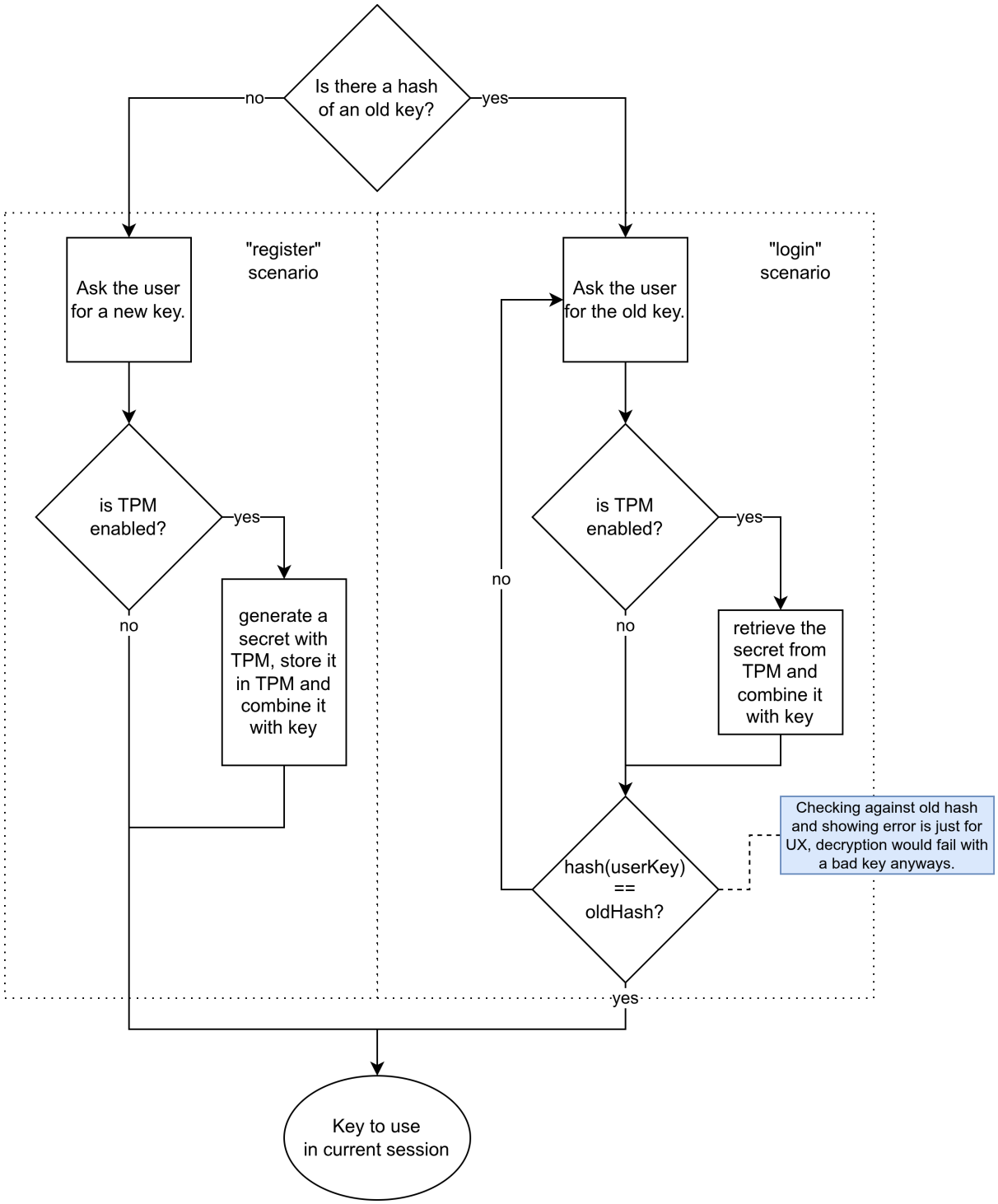


Figure 4.1: The two main flows of the proposed framework.

4.1.1 Disk Encryption

The first line of defense is provided by the device on which the app resides: the USB itself shall be encrypted. I will refer to the password used for the USB's encryption/ decryption as the **disk encryption password**. Each operating system has specific USB formats and default disk encryption algorithms. For Linux, Ext4 partition type encrypted with LUKS2 can be used (see: Section 3.7). For Windows, W95 FAT32 partition type encrypted with BitLocker can be used. The encryption of the device applies on its whole content. This ensures that the stored secrets cannot be read without providing the disk encryption password. Even if the running application saves passwords in plaintext, the USB encryption would encrypt it anyway.

The main issue of USB sticks (in our context) is that they are not read-only media. Someone could steal the USB stick and copy its contents in less than a minute. If the malicious actor puts the USB stick back to where he/she found it, the user would not even know. The attacker could then brute-force breaking the USB without any time limitations. Or even worse, if the attacker gets to know the disk encryption password, he/she can break in without having our media, using the copy. Nevertheless, USB Encryption is a good first line of attack against not-so tech-savvy attackers.

4.1.2 Storage Encryption

Aside of the password for the USB's encryption, the framework also requires a **user password**, for unlocking the application. It is recommended for this password to be different from the password provided for the USB's encryption (see: Section 4.1.1), but a check for difference is out of scope. So it is up to the user which user password it chooses, as long as it matches password complexity requirements. The user password, aside of unlocking the application, is also used to encrypt the TOTP secrets the framework stores, in what we refer to as **storage**.

Of what use is another encryption on top of disk encryption? You might ask, rightfully. Well, disk encryption ensured data safety at rest, while the USB is locked, and storage encryption makes sure secrets are not stored in plaintext, even if the app is running and the USB is unlocked. This protects against "coffee-shop" attacks: the user might leave their device unlocked and unsupervised. In such cases, if the storage would be plaintext, an attacker could quickly read the secrets from their device.

Storage encryption is also a second layer of security on top of disk encryption. While both could be brute-forced, if separate passwords are used for disk and storage encryption, the time required to do so increases. While disk encryption algorithms might be more permissive regarding password complexity, and the framework has no control over it, the user password is required to be complex. It is kind of a zero-trust scenario, where even if the user does not properly encrypt the disk it uses the framework on, the secrets are not leaked.

The user password is never stored. Instead, the hash of the user password is stored inside a file, in what I refer to as the **hashed user password file**. If the hashed user password file does not exist, the framework considers that no user password was provided beforehand. The user is prompted to enter a new password. This is the **register scenario**; although it is a one-user application, it is easier to refer to it with such standard naming. If the hashed user password file exists at start, the framework considers that registering happened beforehand. The user shall provide the old password, which is verified against the existing hash. An incorrect password prevents the start of the flow, since the system

has no knowledge about the password. Without it, storage cannot be decrypted and secrets cannot be retrieved.

4.1.3 Using TPM

The read-write nature of a USB cannot be changed, and making a USB stick uncopiable is either impossible or out of scope. The encryption with the help of the user password suffers the same vulnerability as the USB encryption, it is just another layer, but breakable in the same fashion. Detecting if another TOTP Application is using the secret (for example, if the attacker stole the secret and introduced it into a TOTP application) is also out of scope. However, the time required for brute-forcing can be made even bigger with the help of the Trusted Platform Module (TPM, see: Section 3.8).

The framework proposes generating a random number with the TPM chip, further referred to as the **TPM secret**. Since it is generated with dedicated hardware, it can be trusted (pun intended) to be truly random. Its length shall be considerably large, for example, 16 bytes. The TPM secret can then be combined with the user password in a cryptographically correct/ safe way, to produce what I refer to as the **key** or **key to use in current session**. This shall be used for encrypting and decrypting the storage instead of using the user password directly.

One such combination method of the user password and the TPM secret is through the AES algorithm (see: Section 3.4). The user password runs through PBKDF2 (see: Section 3.5), and the result is used as the initial secret of AES. The TPM secret is long and random, perfect to be used as the salt of AES. Brute-forcing this setup is unfeasible with current technologies.

The TPM secret has to be available on further sessions, since without it, the key cannot be derived. It shall be stored in the TPM's NVRAM. This is non-volatile memory, meaning that it will be present after reboot. When needed, the NVRAM is queried for the TPM secret. The TPM secret is not stored in any form (not even in a hashed version like the user password).

The main drawback of using TPM is the same as its main benefit: it ties the application to a specific machine. If the user uses one machine all the time, this is not an issue. If the user switches between multiple devices (which support USBs), he/she might consider not using TPM. Also, setting up TPM is troublesome and requires technical abilities. For these reasons, the framework marks TPM usage optional. If TPM functionalities are disabled, a hardcoded salt is used instead of a TPM secret.

4.2 Implementation

The implementation of the proposed framework presented in Section 4.1 is the application called Authenstickator. To keep the code modular and maintainable, it follows the Model-View-Controller (MVC) architectural pattern, as depicted in Figure 4.2. The model consists of Python classes handling business logic and everything related, including storage. The view is made with HTML, CSS and vanilla JavaScript. The intermediaries between model and view lie in the controller package, which are Python classes as well.

Authenstickator uses the `pywebview` library to provide a desktop application GUI which is written in web technologies. I chose to write HTML and CSS instead of using native GUI elements for a more uniform and cross-compatible experience. I kept design simple, without going too heavy on JavaScript. There are no JS libraries or frameworks

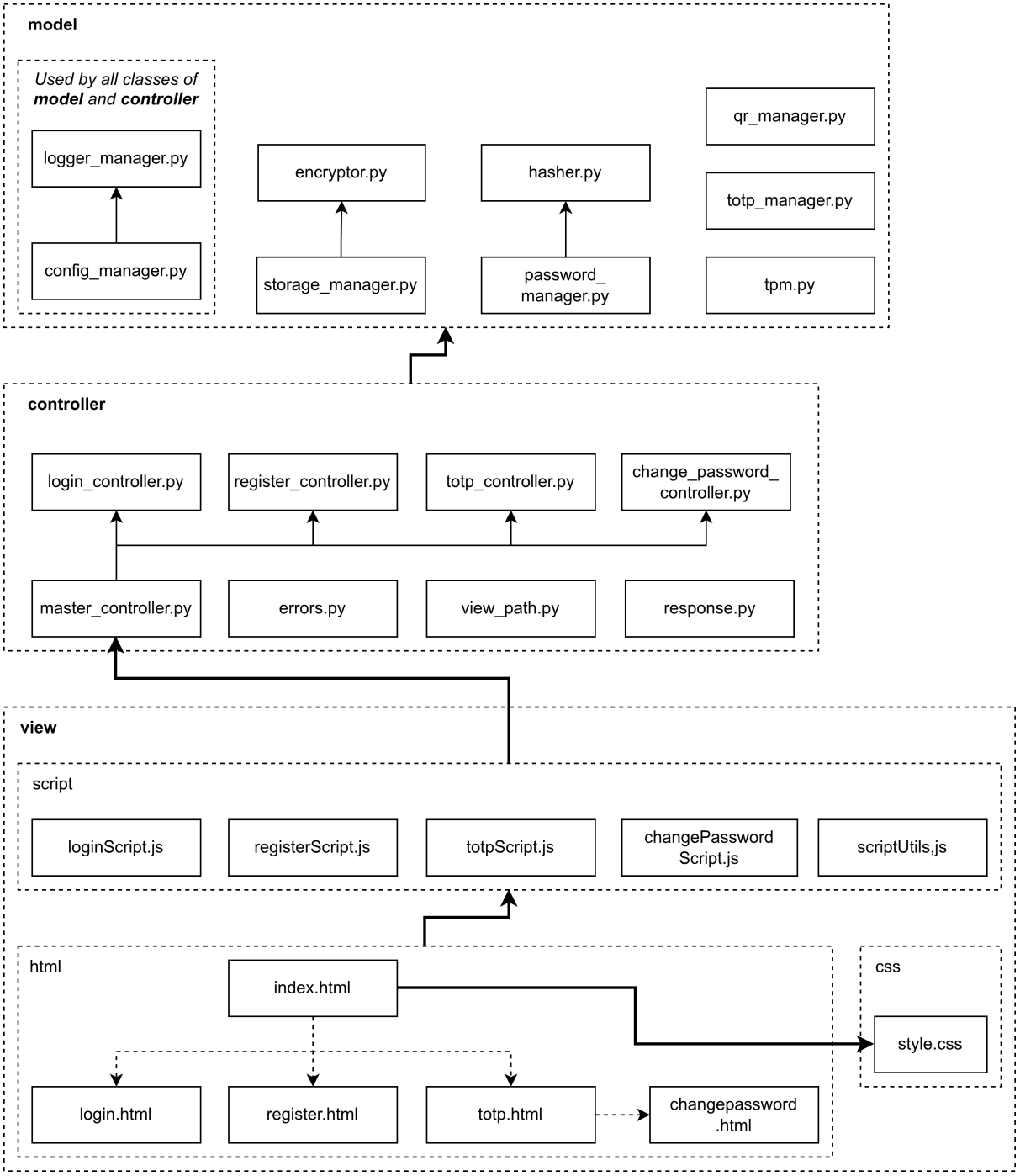


Figure 4.2: High-level diagram of Authenstickator.

included, since that would have a huge impact on the project size and performance. Keep in mind that Authenstickator needs to run on a USB stick, so choosing lightweight solutions is a must.

Each of the packages will be described in detail in the following. When thinking of flows, or when interpreting Figure 4.2, keep in mind that everything is triggered by the actions of the user. Flow always starts from view, where JS triggered by either the user interacting with HTML elements or by periodic method calls/ timers. Then, the JS handler methods delegate to controller handler methods, which orchestrate the classes of model, calling business functionalities and tying them together. The model classes are kept as independent as possible, and they are unit tested.

4.2.1 Model

As mentioned earlier, the back-end of the application is written in Python. While it might sound like a weird choice for a security-first application, there are reasons for this decision. First and foremost, Python provides a lot of libraries, making implementation easier. Using standard implementations and not reinventing the wheel is usually the better approach for small teams. This is even more true if we talk about cryptographic algorithms, like encryption and hashing. Authenstickator's backend contains plenty of these, so developing in Python enabled access to a lot of standard, popular, community-audited libraries.

Second of all, Python provides an easy way to manage dependencies. When implementing Authenstickator, I also focused on making the developer experience easier. The setup of a project is always a big pain in the butt; Python's package manager (`pip`) makes it bearable. All the libraries are downloaded inside a virtual environment (`venv`), so they are isolated and do not mess with the system's packages. The downloaded packages are then listed inside a `requirements.txt` file, so one can install all of them with a single command.

While Python isn't famous about its OOP capabilities, I chose an object-oriented approach. For me and for most developers, it is simply a thing of habit to think of complex problems in terms of classes, objects, and interactions between objects. It provides structure and a common language. In Authenstickator, (almost) every Python file contains exactly one class, kind of like how Java enforces it. Interfaces are implemented using `ABC` and `abstractmethod` from the `abc` library. The overridden methods are marked with the annotation `override` from the `typing_extensions` library. These mechanisms do not cause runtime errors, but generate warnings in the right IDE, making it easier to catch bugs.

Similarly, the `typing` library is used for type hinting. Python is duck-typed, meaning that for Python, if something quacks, it is a duck. "The Python runtime does not enforce function and variable type annotations."[1], but the `typing` library provides runtime support for type hints. The data type of method parameters and the return values of the methods can be specified. Mismatches will generate warnings. Type hints also document intent: it is easier to understand what the method tries to achieve if you can picture the underlying data types.

The backend consists of several packages, which will be presented in the following sections. Each package is self-contained, i.e., independent of the other. As a result, they can be (and are) unit tested (see: Section 5). As a secondary side-effect, this isolation

[1] `https://docs.python.org/3/library/typing.html`

made it so that for most classes it made sense to use the Singleton pattern. The singleton pattern ensures that there is just one instance of a given class at a time.

Singletons, singletons everywhere.

Since I wanted easily manageable classes, and probably since I am influenced by the fact that I worked with Java Beans for the last couple of months, all the classes of model follow the singleton pattern. The main reason is philosophy: these classes encapsulate logic and state which shall be the same over the whole application. Encryption, hashing, TOTP code generation etc. all work the same way, independently of where they will be called. In some cases, static methods would have been enough, but for uniformity, and since static is not always an option, everything is a singleton class.

An example where singleton makes life easier than a pure static design: encryption. While Authenstickator never stores the user password and the salt on disk, they are kept as class fields of singleton `Encryptor`. Yes, this means the secret is stored in RAM, but so it is if it's just read for a split second. Having it as class fields, we do not need to re-prompt the user at each encryption operation, nor to query the TPM for the secret each time.

There are two types of singletons in Authenstickator. The classes with "Manager" in their name are simple singletons, for example, `LoggerManager`. It contains a `instance` field. If the constructor is called and `instance` is not None, it is returned instead of running a new initialization. Else, `instance` is built up by filling its fields, running initialization logic and everything that happens in a constructor, as usual. The seconds type of singletons define a facade for multiple classes. For example, `TPM` is the TPM singleton: it has the usual singleton logic, but it also picks the right TPM class from a pool of multiple possibilities depending on the platform and the configs. Each option implements `AbstractTPM`, which is the common interface. This pattern is used where flexibility is desired: we want a singleton, but we have options for the specific class.

Configurations and how Authenstickator is made parametrized.

Authenstickator has certain parameters which can be changed from a JSON file, without changing source code. The `config` package contains the `ConfigManager`, as well as one or two JSON files. The `ConfigManager` is a singleton class, used by all of the classes of the model and controller. It exposes a method, through which a JSON configuration file is parsed, and the config requested as a parameter is returned. This way, the user of the application can change the behavior of Authenstickator without downloading a new release or changing the source code.

The name of the config file is hardcoded, and it is `config.json`. It has multiple levels, as show in Listing 4.1. Developers also have the possibility to define a `config-local.json` file, which, if present, has priority over `config.json`. The local config file shall be in `.gitignore` and used solely for local development.

Listing 4.1: An example configuration file.

```
{
  "logger": {
    "log_level": "DEBUG",
    "log_file_path": "log.txt"
  },
  "encryptor": {
```

```
    "type": "AES"
  },
  "hasher": {
    "type": "ARGON2"
  },
  "password": {
    "min_length": 15,
    "max_length": 100,
    "hash_file_path": "hash.txt"
  },
  "storage": {
    "storage_file_path": "storage.txt"
  },
  "pywebview": {
    "debug": false,
    "width": 450,
    "height": 800
  },
  "tpm": {
    "enabled": false,
    "virtual": {
      "fapi_profile": "P_RSA2048SHA256",
      "fapi_profile_url": "https://raw.githubusercontent.com/tpm2-
       software/tpm2-tss/3.2.0/dist/fapi-profiles/P_RSA2048SHA256.json"
       ,
      "enabled": false,
      "tpm_path": "/tmp/vtpm",
      "tpm_port": 2321,
      "tpm_ctrl_port": 2322,
      "tpm_start_wait_s": 1
    }
  }
}
```

The meaning of each parameter is the following:

- `logger.log_level`: The log level to use for the one and only logger. Everything higher or equal will be logged, everything lower will be ignored. Accepted options: `DEBUG`, `INFO`, `WARNING` `ERROR`.
- `logger.log_file_path`: The path of the log file. Logs are written to console but also to a log file, for audit, troubleshooting, and for cases there a console is not available.
- `encryptor.type`: The encryptor to use. As of v1, only `AES` is supported.
- `hasher.type`: The hasher to use. As of v1, only `ARGON2` is supported.
- `password.min_length`: The minimum length required for user password, inclusive.
- `password.max_length`: The maximum length accepted for user password, inclusive.
- `password.hash_file_path`: The path to the hashed user password file.
- `storage.storage_file_path`: The path to the storage file.
- `pywebview.debug`: If set, pywebview's debug mode is enabled. Accepted options: `True`, `False`.

- `pywebview.width`: The width of the pywebview window.
- `pywebview.height`: The height of the pywebview window.
- `tpm.enabled`: If set, TPM functionalities are enabled. Accepted options: `True`, `False`.
- `tpm.virtual.fapi_profile`: Which FAPI profile to use for the virtual TPM. Will be passed to `fapi-config.json`, and used as file name for the FAPI profile file. Must be in sync with parameter `tpm.virtual.fapi_profile_url`.
- `tpm.virtual.fapi_profile_url`: The URL from which the FAPI profile can be downloaded for the virtual TPM. Must be in sync with parameter `tpm.virtual.fapi_profile`.
- `tpm.virtual.enabled`: If set, the software TPM is used instead the real TPM chip. Setup will run based on other `tpm.virtual` parameters. Accepted options: `True`, `False`.
- `tpm.virtual.tpm_path`: Since TPMs on Linux are device nodes, this is the path to use for the virtual TPM.
- `tpm.virtual.tpm_port`: The port to use for the virtual TPM.
- `tpm.virtual.tpm_ctrl_port`: The control port to use for the virtual TPM.
- `tpm_start_wait_s`: The amount of seconds to wait before checking if the virtual TPM got successfully provisioned, after calling the provision function.

How logging is done.

Logging is done by the package `Logger`. It contains the `LoggerManager` class, yet another singleton. As its name suggests, the logger manager provides a logger instance. It is the one and only way to print messages, the print statement and similar shall not be used.

The standard `logging` library is used. The log level can be from config (see: Section 4.2.1). For simplicity, Authenstickator uses only four log levels. It is important to select the right log level each time, otherwise important logs might get cluttered or not printed with certain configs. From most severe to the least, these are:

1. **Error**. For fatal issues, ones which break execution.
2. **Warning**. For issues which should not be ignored, but do not crash the app.
3. **Info**. For messages which are not issues, but things that might help in audit.
4. **Debug**. The most low-level messages. These are useful only for development.

While the logger is centralized, logging happens in two places: the console and the log file. The same log lines are printed in both outputs, but the log file persists, meaning that new logs are appended to the old ones instead of starting a new file at each run. This is helpful for debugging and provides a possibility to audit previous actions, for example, if someone else tried to log in.

The format of log lines is: date and time, severity level in a letter (E, W, I, D), the file and the line of the source, the logger, and the message itself. Some log lines are shown on Figure 4.3. While there is only one logger instance, `pywebview`, the front-end library also logs its warnings and errors. These are useful, so instead of throwing them out, they are "propagated" to `LoggerManager`'s instance. This is why the logger is also printed in the log line: it can be either `[root]` or `[pywebview]`.

```
1757
1758
1759 AUTHENSTICKATOR
1760
1761
1762 [2026-08-29 09:53:18] [W] [storage_manager.py:46] [root] File storage.txt not found, no secrets loaded.
1763 [2026-08-29 09:53:23] [D] [storage_manager.py:105] [root] Storage saved.
1764 [2026-08-29 09:54:51] [I] [logger_manager.py:59] [root] Logger configured. Starting the application.
1765 [2026-08-29 09:54:51] [I] [logger_manager.py:61] [root]
```

Figure 4.3: Example log lines from the log file.

As a general rule, secrets and sensitive data shall never be logged. Logging such information be the equivalent of a data leak.

Generating TOTP codes.

Class `TOTPManager` is responsible for generating the TOTP codes, as well as other TOTP-related functionalities. The `pyotp` library was used, which is straight-forward and provided every functionality I needed. As a matter of fact, there are only two such functionalities: generating TOTP-related information and parsing TOTP URIs.

By **TOTP-related information**, I mean the current six-digit code, the next six-digit code, when the current code expires, and when the next code expires, as well as the interval. The **interval** is the time for which one TOTP code is valid. It is usually set to 30 seconds, as recommended by the authors of the TOTP algorithm.[7] However, Authenstickator does not assume it is 30 seconds all the time, instead it deduces it from the secret. Deducing the info from the secret is as simple as calling a constructor and then extracting its fields, although some mathematics had to be done.

Counterintuitively, I do not send to the frontend the seconds remaining. This would be a huge overhead, just think about it: querying every second for every secret, just to get back the same redundant information roughly 58 times a minute. Instead of querying each second, I could just automatically send each second, but again, this is a lot of redundancy, and it does get hacky. Instead, what I chose to do, is to send the expiration times to frontend, and let the frontend calculate the remaining time. Some kind of loop/ repetitive check is needed anyway, and a frontend timer is probably the best solution. The frontend can then query for fresh information when the current code expires.

But what happens, if the querying takes longer than expected? Or if the number of codes to query for is large, and they expire at the same time, causing an unexpected overhead? Actually, the latter is rather expected, as almost all apps use the default TOTP parameters, and hence their codes all expire at the same moment (I have not seen any counter-examples yet). Well, this is why I also send the next code. It is trivial, and the TOTP algorithms allows us, to retrieve the next code. In fact, any code could be computed, since the code is a function of the current time. There are authenticator apps, like Proton Authenticator, which also show the next code. So, the frontend has

the current code and also the next one: when the current code expires, it can instantly **promote** the next code, i.e., make it current. This results in a smooth transition.

Parsing the TOTP URI means extracting information from the URI which the apps usually show in QR form. Yes, those fancy QR codes are nothing more than just a "link", which usually/hopefully follows the Key Uri Format specified by Google[2]. In the `TestTOTP` test class you can see some examples of such URIs, as different platforms provided to me:

1. `otpauth://totp/totp@authenticationtest.com?secret=I65VU7K5ZQL7WB4E`
2. `otpauth://totp/Google%3Arolandtest123456%40gmail.com?secret=refyopv7thgvsd5twr3marvdpqyugxls&issuer=Google`
3. `otpauth://totp/Microsoft:rolandtest123456@gmail.com?secret=G5HLROANXBF7335S&issuer=Microsoft`

The first one comes from the test site `https://authenticationtest.com/totpChallenge/`, which I used a lot throughout development. It provides a hardcoded secret, both in plaintext and in QR form. The QR, when parsed, resulted the URI shown with number 1. Comparing it with the Key Uri Format examples, you can see that it contains the label `totp@authenticationtest.com`, and the secret `I65VU7K5ZQL7WB4E`. It does not contain the issuer, although it is "STRONGLY RECOMMENDED" by Google, it is not required, hence we cannot assume that we get it all the time. The second URI comes from a dummy Google account I made. You can see that it used URL encoding[3]: `%3A` $\rightarrow$ `:`, `%40` $\rightarrow$ `@`. The label also contains the issuer. The third URI is from a dummy Microsoft account. Similarly to number 2, this one also specifies an issuer, and the label already contains the issuer.

In Authenstickator, storage contains just two information: the secret and the name. The name is combined as `ISSUER:LABEL`, if both available, or just the label, if it already contains the issuer, or just label, if issuer is not provided.

Decoding QR images.

Every mobile authenticator app provides the possibility to scan the QR code provided by the app which requires MFA. But, a USB stick does not have a camera. Initially, the only way to enter a TOTP secret into Authenstickator's database was to enter the secret manually, enter a name manually, and press enter. But, what if the app only shows a QR, and does not give us the plaintext equivalent? In that case, we could use an online tool to decode the QR image, but this would mean that we blindly trust a random website with handling out sensitive information.

As a solution, I integrated QR decoding functionality into Authenstickator. It requires an image of the QR code, so a snip has to be saved beforehand. **The user shall delete the screenshot after decoding it, or store it in a secure location.** Otherwise, the secret could be exposed. The app could delete the image after decoding, but it leaves it to the user; who am I to manipulate your files? It would kind of break the isolation idea of Authenstickator, according to which the territory of Authenstickator is only the USB stick.

[2] `https://github.com/google/google-authenticator/wiki/Key-Uri-Format`
[3] `https://www.calculator.net/url-encode-decode.html`

The QR code image is decoded using the `zxingcpp` library. Initially, `pyzbar` was used, but it brought extra requirements for Windows packs, so I switched it to `zxingcpp`, which is newer and still maintained. With the `pillow` library I opened the image, and with the `zxingcpp.read_barcodes(image)` call I got the decoded text. The text is a URI which is passed to TOTP for parsing (see: Section 4.2.1). If there are no QR codes found in the image, I return None. If there are multiple QR codes, the first one is processed, and a warning is logged.

Hashing and checking against a previous hash.

Authenstickator currently uses the Argon2 algorithm (see: Section 3.6) in its `Argon2Hasher` class for hashing purposes. The `argon2-cffi` library is used. It provides a hashing and a verifying method. It is worth noting that the same input creates different hashes, so verification is a library function call as well, not hashing and a simple equals operation. The latter would not give correct results.

Policies for the user password.

The `PasswordManager` class implements the password policies for Authenstickator. It is generally a good practice to extract password management into a separate class or module, to apply the same rules everywhere. This is the only class which uses the user password hash file, no other classes shall read from or write to it, or modify it in any way. The class exposes a method for checking if a previous password was provided, which does nothing more than check if the user password hash file exists. This method determines upon startup if we go through the register or through the login scenario.

The password manager is also responsible for validating new passwords. If they satisfy complexity requirements, it is said that they are "acceptable". An acceptable password in Authenstickator has a length between a minimum and a maximum length specified by configs (see: Section 4.2.1). But, since Authenstickator takes security seriously, if the minimum length parameter is set to less than 15, it is switched to 15. This is to follow the guidelines published by NIST SP 800-63B-4[4], Section 3.1.1, which says: "Verifiers and CSPs SHALL require passwords that are used as a single-factor authentication mechanism to be a minimum of 15 characters in length."

There is no special character requirement and such craziness. We are well past the age where special characters provided any benefit. Nowadays, they are rather an obstacle, in defining long a memorable passphrase. The same NIST article, in its Appendix called Strength of Passwords, doubles down on this idea, saying that:

- "Password length is a primary factor in characterizing password strength"
- "Research has shown that users respond in very predictable ways to the requirements imposed by composition rules. For example, a user who might have chosen "password" as their password would be relatively likely to choose "Password1" if required to include an uppercase letter and a number or "Password1!" if a symbol is also required."
- "Other mitigations (e.g., blocklists, secure hashed storage, machine-generated random passwords, rate limiting) are more effective at preventing modern brute-force attacks, so no additional password requirements are imposed."

[4] `https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63B-4.pdf`

The password manager uses the hasher (see: Section 4.2.1) to hash the user password before saving it to the user password hash file, and to verify the correctness of a provided user password, by reading the hash and verifying against it. Setting a new password implicitly involves checking if the new password is acceptable. There is also a hard limit of 100 maximum characters for the password, but this is just a sanity check, to avoid crazy users causing an overflow.

TPM and the generation of the TPM secret.

Authenstickator currently supports TPM functionalities only on Linux (see: Section 6.1). Using the second singleton approach described in Section 4.2.1, the `TPM` class chooses between `NoTPM`, `LinuxTPM` and `WinowsTPM`, where `WindowsTPM` contains identical implementation to `NoTPM`. If TPM functionalities are disabled from configs (see: Section 4.2.1), `NoTPM` is chosen. Otherwise, the current platform dictates what to do. `LinuxTPM` also provides support and setup for a virtual TPM, for development purposes.

As explained in [15], on Linux, TPM shows up as device nodes. `/dev/tmp0` is the Hardware Interface, while `/dev/tpmtm0` is the Kernel Resource Manager. Most applications use the Kernel Resource Manager, since through that entry the kernel handles context management and other low-level mechanisms.

TSS stands for Trusted Software Stack, and it is the software around the TPM chip. It has a layered architecture, providing programmer-friendly API calls. Its layers are specifications, and there are different implementations of it. The four layers are:

1. **FAPI**: Feature API. High-level, developer-friendly. No need to handle sessions and authorization.
2. **ESAPI**: Enhanced System API. Wraps sessions and handles retries when TPM is busy.
3. **SAPI**: System API. Low-level C interface implementing the tmp2.0 specifications.
4. **TCTI**: TPM Command Transmission Interface. The Transport Layer, working with bytes.

Authenstickator uses the `tpm2_pytss` library and the FAPI layer. FAPI was chosen since it is the most high-level, so the resulting code will be more generic. Going lower in the TSS layers would mean more OS/ hardware specific code, and more chances to make errors.

When this layer is used, configurations are held in a file named `fapi-config.json`. The default location is `/etc/tpm2-tss/fapi-config.json`, but it can be overwritten by setting the `TSS2_FAPICONF` environment variable. This file is generated automatically when you install or compile the libraries for FAPI provisioning, and it looks as shown in Listing 4.2. **Provisioning** is an initialization step necessary to be performed before using FAPI. The TPM, FAPI and the related files and configs are not well documented, so most of what I write here are the results of trial and error.

Listing 4.2: The default FAPI configuration file, as on my system.

```
{
  "profile_name": "P_ECCP256SHA256",
  "profile_dir": "/etc/tpm2-tss/fapi-profiles/",
  "user_dir": "~/.local/share/tpm2-tss/user/keystore",
```

```
  "system_dir": "/var/lib/tpm2-tss/system/keystore",
  "tcti": "",
  "system_pcrs": [],
  "log_dir": "/var/run/tpm2-tss/eventlog/",
  "firmware_log_file": "/sys/kernel/security/tpm0/
   binary_bios_measurements",
  "ima_log_file": "/sys/kernel/security/ima/binary_runtime_measurements
   ",
  "ek_cert_less": "yes"
}
```

Provisioning a real TPM might require authorization, or might have been done beforehand is there is other software using FAPI. This is a tedious task which might differ from hardware to hardware, includes many edge cases and possible error sources. I generated with the help of AI a setup script for provisioning, which worked on a fresh Ubuntu 22.04 installation, see Section 4.3.4. You can try it, or do it manually, but note that a provisioned FAPI is a prerequisite if you select non-virtualized TPM from configs.

For development, but strictly only for that, a virtual TPM is encouraged, which can be opted for through the configs. By using a virtual TPM, you won't get locked out of the real TPM if you repeatedly violate authorization. A virtual TPM can be wiped completely anytime, and a new instance can be started. For the virtualization of the TPM, the `swtpm` Linux library is used. The `LinuxTPM` class contains code for setup which should work out of the box, running each time if you selected virtualization. It creates a custom `fapi-config.json`, as seen in Listing 4.3.

Listing 4.3: The virtual FAPI configuration file, as on my system

```
{
    "profile_name": "P_RSA2048SHA256",
    "profile_dir": "/home/broland/.local/share/authenstickator/profile-
     dir",
    "user_dir": "/home/broland/.local/share/authenstickator/user-dir",
    "system_dir": "/home/broland/.local/share/authenstickator/system-
     dir",
    "tcti": "swtpm:port=2321",
    "system_pcrs": [],
    "log_dir": "/home/broland/.local/share/authenstickator",
    "ek_cert_less": "yes"
}
```

You can note the slight differences: there is a port defined, and a different profile is used. The FAPI profile is downloaded from the internet, from the URL specified in the config file. The profile's version shall match the version of the installed native `tpm2-tss/libtss2-fapi`. This can be retrieved with the help of the command `apt policy libtss2-fapi1`. Note that this is different from the `tpm2-pytss` version shown in `requirements.txt`.

If configuration, provisioning, and all the setup steps are done correctly and successfully, the actual usage of the TPM through FAPI reduces to just a few lines. These high-level functions provide convenient wrappers for TPM functionalities. The random number (TPM secret) is generated with `fapi.get_random(16)`, where 16 means the secret will be of 16 bytes length. This number is then saved to and later retrieved from the TPM's Non-Volatile Random Access Memory (NVRAM), using the `fapi.create_nv`, `fapi.nv_write` and `fapi.nv_read`. For each method, I used the same path, which basically defined the slot of NVRAM used, but abstracted, without going to index and byte

level. The existence of the TPM secret before generation is checked with `fapi.list("/")`, which gets a list of all FAPI object paths. If the result contains our path, it means the secret was generated.

The FAPI method `create_nv` is called with the flag `exists_ok=True`, and `provision` (for the virtual TPM) is called with flag `is_provisioned_ok=True`. Both of these, in the case of previous create/provisioning, write an error in the log files, but do not actually cause crash. This is because `tpm2-pytss` is a wrapper for the `tpm2-tss` library written in C, and this is the best the wrapper's authors could come up, I guess. This is a beauty flaw of the logs, but they do not mean actual errors.

Encryption and decryption.

While the encryptor singleton allows the existence of multiple encryptors, currently only `AESEncryptor` is implemented. As its name suggests, it uses the AES algorithm (see: Section 3.4). The user password and the TPM secret is run through the PBKDF2 algorithm (see: Section 3.5). This basically combines the user password, which could be of any length (between the limits imposed by the configs), and the TPM secret, which is used as salt, into a key of 32 bytes length. The result can be used as a key for the AES algorithm. AES is a symmetric algorithm, so the same key is used for encryption and decryption.

The libraries `Crypto.Cipher`, `Crypto.Hash` and `Crypto.Protocol.KDF` are used for `AES`, `SHA256` and `PBKDF2`, respectively. For the PBKDF2 algorithm, I used 600000 iterations, with SHA256, as specified by the OWASP Password Storage Cheat Sheet[5]. This is a work factor, and having a high value means it is harder to brute-force, but slower for malign use as well (it is "intentional slowness"). So instead of balancing between security and usability with random trial and error, I consider that following trusted guidelines is a better option.

This AES library works with triplets of nonce, tag and ciphertext, i.e., the AES-EAX variant of the algorithm. In practice, this means that at each encryption I create a new cipher using `AES.new`. The ciphertext returned by the `AESEncryptor`'s `encrypt` method will then contain the nonce, concatenated with the tag and the actual ciphertext. At decryption, this triplet is split. A new cipher is created with the same key and the extracted nonce, and the tag is passed to `decrypt_and_verify` to ensure correctness. This approach avoids the common pitfall of using the same nonce twice, which often turns such algorithms "from gold-standard to broken" (see [6] for an article which describes AES-GCM, which is another variant but at high level api point of view works the same way).

The `AESEncryptor` and its respective singleton class, `Encryptor`, went through multiple refactorings throughout development. The final version does not set the key in the `__init__` method, but has a dedicated `set_key` method. This is because the user password can change: Authenstickator supports password reset. In such cases, we want to change the key, but we do not want to swap out the singleton instance. The `set_key` solves this exact issue: the key (internal state) is changed, but the instance (the reference) remains the same, so the change is "implicitly propagated" throughout the code base.

[5] `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html#pbkdf2`

[6] `https://www.trustedcalcs.com/en-us/guides/dev-tools/aes-gcm-encryption`

How the storage file is handled.

The `StorageManager` class is the one responsible for handling storage. Storage means the "database", which in practice is a single file, called the storage file. The storage file's path can be defined as a config (see: Section 4.2.1). Storage manager uses the encryptor (see: Section 4.2.1) to decrypt the storage file upon initialization. If encryption fails (either the key is wrong, although password checking should guard against this, or the file is broken, against which Authenstickator cannot protect), storage initialization is considered failed, and the `__new__` method returns None. This has to be handled by the caller.

After successful decryption of the storage, the result is a JSON file which maps to a one-layer dict, where the key is the name (as described in Section 4.2.1), and the value is the TOTP secret. The choice is intentional and would not work in reverse: the secret is provided by the user once, after which the UI only holds reference to the name, hence queries by name. Keeping the secret in the UI would mean an information leak. The fact that the underlying data structure is so simple makes saving trivial, symmetric to retrieval: the internal storage is dumped (`json.dumps`), the dump is encrypted, and the ciphertext is written to the file.

The class exposes methods for adding a secret, removing a secret, getting a secret. There is also a "get all secrets" method, which shall be used at startup to load all the data. These methods have straight-forward validations: an entry cannot be added if the name is already in use, an entry cannot be removed and/or retrieved if it does not exist.

Initially, I wrote the `StorageManager` to support the Context Manager Protocol, i.e., such that it can be used with Python's `with` syntax. This meant that `StorageManager` was used inside a `with` block, and when execution got out of it in any form (the instructions were all executed or an exception happened), a cleanup function was called. However, I found an even stronger and more robust solution: just save each time we modify storage. This leaves no space (as in time window of intermediary instructions) for things to mess with storage. If a secret is successfully added, the file is saved. If a secret is successfully deleted, the file is saved. This might sound like a huge overhead, but it is not: adding and removing secrets is not a frequent operation. And, let's be real, how many TOTP secrets can a user have at most?

4.2.2 View

The view module of the application was implemented in CSS, HTML and vanilla JavaScript. Although these are technologies usually associated with web applications, Authenstickator is completely offline. It uses the `pywebview` library to make this possible. "pywebview is a lightweight BSD-licensed cross-platform wrapper around a webview component. pywebview allows to display HTML content in its own native GUI window. It gives you power of web technologies in your desktop application, hiding the fact that GUI is browser based. pywebview ships with a built-in HTTP server, DOM support in Python and window management functionality."[7].

I wanted to avoid choosing a niche technology, and I also wanted something lightweight and simple. For the purpose of Authenstickator, standard elements were needed, mostly just inputs, labels, and buttons. The fact that I chose HTML and CSS for the view made it such that the code base is easy to understand, universal, so to say.

[7] `https://pywebview.flowrl.com/`

JavaScript was needed to intercept user actions, but the view is not completely "dumb". For example, it calculates the remaining time until TOTP code expiration, since it is more efficient to do this on the front-end. It also periodically calls the backend for retrieving new TOTP codes. So, as long as there is a reason to have it there, and it does not handle sensitive information, logic is also put inside JavaScript methods.

How HTML files are structured.

Instead of having one huge index.html and a big spaghetti code, I chose to define different views and embed them inside index.html. These views are defined based on what the intent of the user is when it is on them, namely: `register.html`, `login.html`, `totp.html`, and `changepassword.html`. This way, the main file (`index.html`) becomes just a container, the holder of the `viewDiv`. Initially, this div contains the text `Loading...`, but immediately after loading the app, it is switched to either `register.html` or `login.html`.

The `index.html` file links the CSS file and the scripts (JavaScript files) inside its `head` block. It also displays the big "Authenstickator" text, which is permanent on each view. Inside a `script` block, there are two globals defined, `RESPONSE` and `VIEW_PATH`, but more on this later. The most important point is that here lies an event listener which catches when the view is loaded (`pywebviewready` envent), after which the JS API is called, and execution gets under control. From this point on, depending on user actions, the views will be changed.

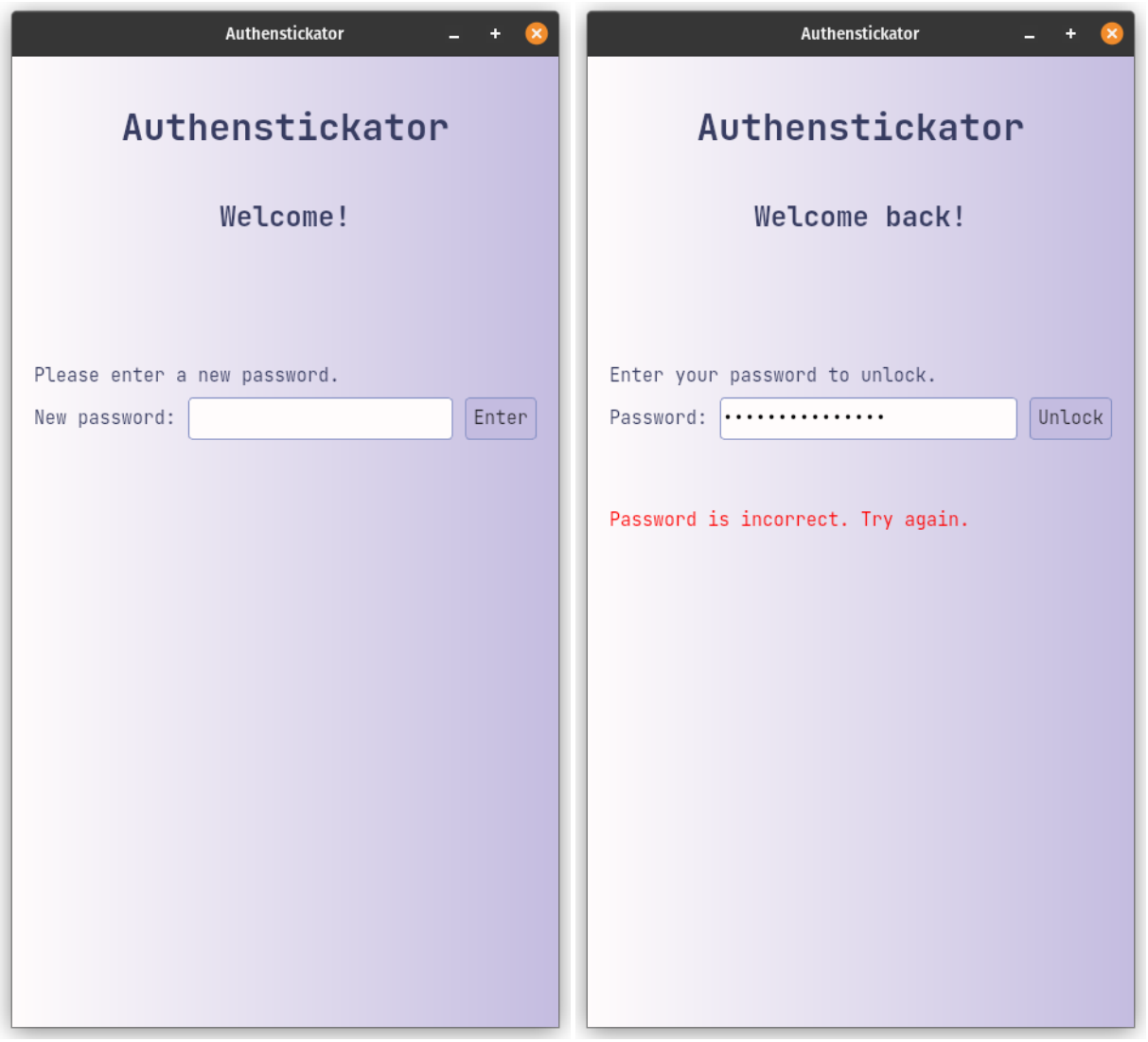
The views and their elements.

The very first view a user will see is the register view (see: Figure 4.4a). `register.html` contains an input for a new password. Focus is automatically put inside this input, so the user can start typing instantly. There is an enter button, which is also clicked if the user is inside the input and presses the enter button. The maximum characters allowed inside the input is limited.

If there was a password provided beforehand, the login view will appear instead of the register view (see: Figure 4.4b). This is almost identical in shape, but the labels and the intent differs. This time, if enter is pressed or the unlock button is pressed, the password is verified against the old password's hash. If it does not match, an error message is displayed. Error messages appear with red color, while success messages appear with green. Both are displayed inside `messageParagraph`, a `p` element which shall be present on each view. There is no wiping mechanism or anything, the next message simply overwrites the current message each time.

In case of success, both from the register and the login view we get to the TOTP view (see: Figure 4.5a). This is the main view of the application, since this is where the actual functionalities reside. At the top, the user has the ability to enter a TOTP secret in two ways: either manually of through decoding a QR. To add manually, the user has to enter the TOTP secret into the secret field, and also give it a unique name in the name field, after which, the Add button shall be clicked. If there is a secret which was added previously and has the same name as the one the user wants to add, an error message is shown. The secret is also checked to be valid (see: Section 4.2.1).

Adding a secret through QR image decoding might be the only option, if the secret is not provided in plaintext. In this case, the user has to press the QR button, which opens up the default file navigator application, and lets the user choose a file with the



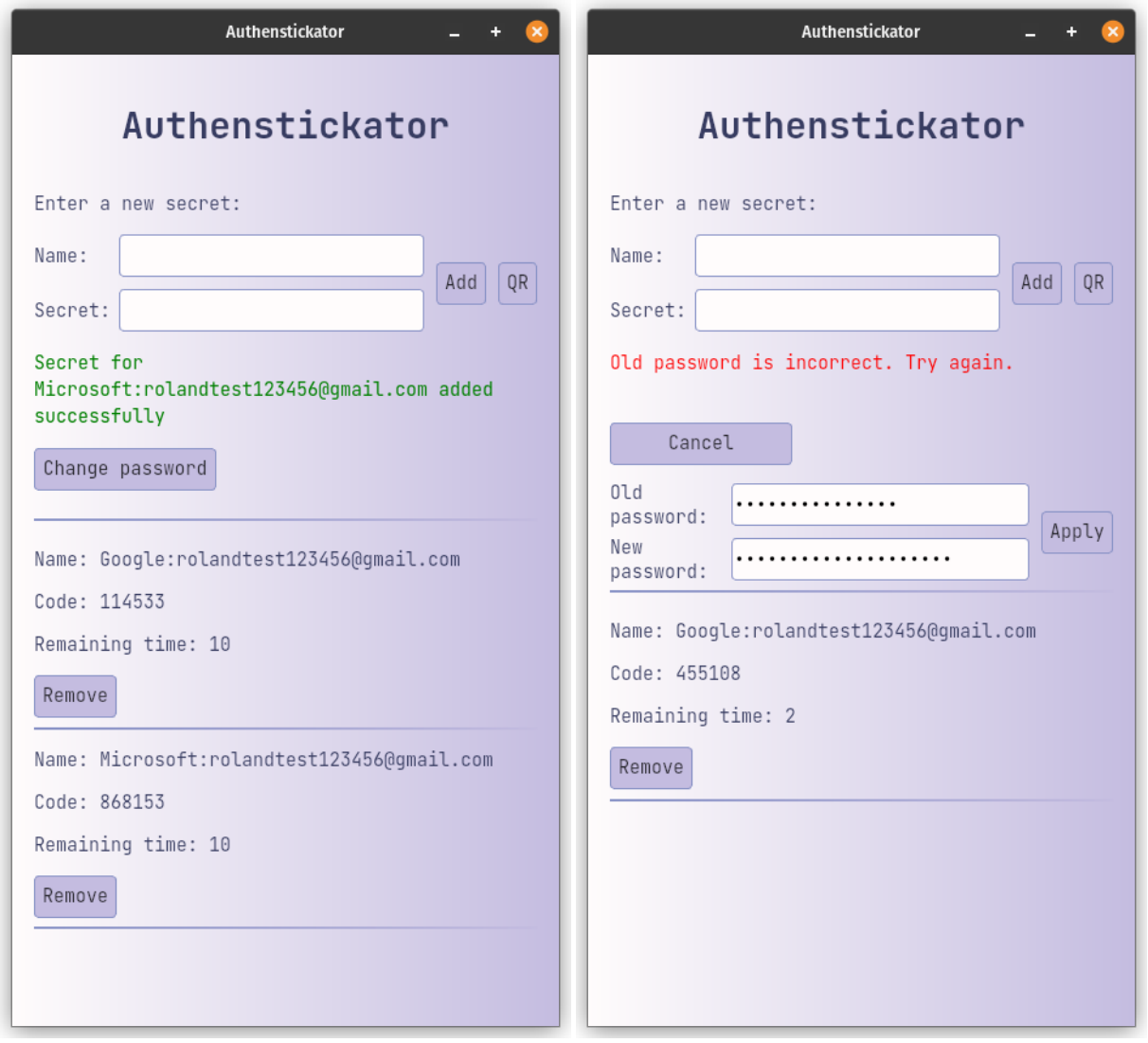
(a) Register view.

(b) Login view.

Figure 4.4: The first thing the user sees is either the register or the login view.

extension `.bmp`, `.jpg`, `.jpeg`, `.png` or `.gif`. This image is parsed, and an error is shown if the parsing was unsuccessful. A successful decoding gives a URI which contains the secret and the name as well (see: Section 4.2.1), so the user has nothing else to do. The extracted name and secret are subject to the same validations as the manually added ones.

One added secret appears after the (somewhat) static "header" as an entry, a new row. If we get to multiple entries, the scrollbar appears, since the view gets longer than the screen. It resembles a mobile authenticator app. I even choose the aspect ratio to be somewhat similar to what a user would see on a mobile device. These entries are not ordered in any way, they are just added at the end of the list, to avoid jumping screens. If the user presses the remove button, the row is removed. Each row shows the name, the code, and the remaining time, in seconds. The remaining seconds are decremented each second.



(a) TOTP view.

(b) Change password view.

Figure 4.5: The change password view is embedded into the TOTP view.

The change password view is embedded inside the TOTP view (see: Figure 4.5b). This design choice comes from the desire to keeps things simple. If the change password

button is clicked, the change password view's elements appear, and the button's text is changed to "Cancel". For a password change, the user has to provide the old password and the new password. The old password provided is verified against the password hash file, while the new password is subject to the same criteria as the password provided at registration (see: Section 4.2.1).

The revealing and hiding of the change password view does cause slight inconvenience in the sense that the TOTP entries/rows are pushed down when revealed and up when hidden. However, password changing is not a frequent operation, and always showing these elements would just take up screen surface. Another option would have been a dedicated view, but this would have brought larger overhead on the logic. Currently, there is no going back mechanism, the views just go in one flow, either login → TOTP or register → TOTP. Also, the actual solution has the advantage that TOTP codes are visible even when having the change password view shown. A popup could have been another option, but again, this is technically hard to pull off, since we talk about a web-based application, not a native desktop app where we simply call a function which launches another view.

Styling, CSS and other resources.

There is a single CSS file for the whole project, called `style.css`. A uniform styling is used for all elements: each button, label, and input looks the same way. I choose a color palette from the website Color Hunt[8], and I adapted the colors a little bit. As a result, four colors are used throughout Authenstickator, which have the color codes `#fffcfc`, `#c4bce0`, `#8892c6` and `#373c61`, which were defined as css constants `--color-1`, `--color-2`, `--color-3` and `--color-4`. The background is a linear gradient from `--color-1` to `--color-2`. For the text, the darkest color is used. The input and button borders are uniform as well, they use `--color-3`. I wanted to avoid white/ light text on dark backgrounds, so the buttons have `--color-2` as the background. The inputs have the brightest color as their background, which is almost white, but less eye-striking.

I also added a bit of responsiveness to buttons. When the user hovers over them, the border gets darker. The transition from the usual border color to the darker color happens over a period of time, not just instantly. When they are clicked, they become smaller for a split second, giving some feedback to the user that the button was indeed pushed.

For the alignment of the elements, I used display flex on the `div`s. The body's CSS specifies the attributes `display: flex`, `flex-direction:column` and `min-height:100%`. This way, the body occupies the whole available space, and from that point on, we think of the UI as stacking rows on each other. The most common row I used is specified as `.inline-form`, with attributes `display:flex` and `align-items: center`. For example, in `register.html`, a `div` with the previously mentioned class contains the new password label, new password input and new password button. In this case, and in most cases, I put the `stretch-input` class on the input, which defines `flex-grow: 1` and `width: auto;`, i.e., the input takes up all the remaining space the label and the button leaves for it. This works only if each `div` has the flex attribute, it does not implicitly cascade.

The font I chose for the application is JetBrains Mono, an open-source font available at `https://fonts.google.com/specimen/JetBrains+Mono`. I put in the `res` folder

[8] `https://colorhunt.co/palette/f4eeffdcd6f7a6b1e1424874`

the `.tff` files for regular, bold and italic, although currently I only used regular. This font looks nice, is monospaced, meaning that each character has the same length. A lot of monospaced fonts look ugly on a UI, they are more popular inside terminals and code editors, but I opted for one such font probably since I am a perfectionist. The JetBrains font looks friendly on a GUI as well, it is rounded and clearly visible.

The application also has a logo, as seen on Figure 4.6. I made by hand in `drawio` and colored it in `MS Paint`. The colors of the application were reused. The logo depicts a USB stick with the text "AUTH" scratched on it. This image is also included as a `.ico` file in the project's `res` folder, used as the icon for the Windows pack. Unfortunately, on Linux, it was not so straight-forward to set the logo of the executable. Software usually uses a `.desktop` file to specify the image which should appear as the icon of the executable, but one such file contains absolute paths. Since we run the software from a USB stick, I cannot specify such a path. So, the Linux pack ships without an icon, but the icon is included in the resources, and the user, if he/ she has the necessary technical skills, can define a `.desktop` file using it.

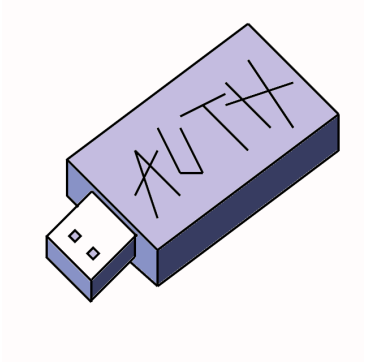


Figure 4.6: The logo of Authenstickator.

The JavaScript giving it life.

As shown in the high-level diagram (see: Figure 4.2), there is a dedicated package for the JavaScript files called `script`. Following the same principle as discussed in Section 4.2.2, each view has a dedicated JS file: `registerScript.js`, `loginScript.js`, `totpScript.js`, and `changePasswordSciptt.js`. Common/ reused functions can be found in `scriptUtils.js`. Each of these scripts is specified in `index.html`. I did not employ classes, they are all in the same namespace, so unique method names are required throughout all these files.

In script utils, we can find the most frequently used method called `callApi`. This is a wrapper for calling an API function, as it will be presented in section 4.2.3. If the silent flag is set to False, this method will clear the previous message (displayed on `messageParagraph`), and display the error message with red, or in success scenario, the success message in green, if it exists. By using this wrapper all the time, I avoided code repetition and ensured that behavior is uniform for each API call. Script utils also contain the `loadView` function, which fetches the HTML from the path specified, adds event listeners for the fetched view, if necessary, and calls the `initTOTP` method of `totpScript` if the view is changed to the TOTP view.

The `initTOTP` method calls the API to fetch all the data from storage and starts the TOTP timer. The timer is implemented using the stock method `setInterval`. It

calls the `updateTimers` method each second, which:

1. Iterates through each entry. For each of the entries then performs the next steps.
2. Gets the expiration milliseconds from the row's dataset (set previously). From the expiration milliseconds and the current date, calculates the remaining milliseconds.
3. If the remaining milliseconds is (less than or) equal to zero, promotes the next code to current code; after which calls the API for getting the next code.
4. Updates the label which shows the remaining milliseconds.

This is how the main functionality of Authenstickator comes to life. The rest of the script files contain the necessary JavaScript to call APIs for handling user actions described in Section 4.2.2. These mostly include button click handlers. One interesting fact is that even though the `fetch` method is universal, it behaved differently on Linux and Windows. On Linux, if successful, it returned `response.ok=false` and `response.status=0`. On Windows, if successful, it returned `response.ok=true` and `response.status=200`. This is probably because on Linux the `file://` protocol was picked, while on Windows it seems like HTTP was the automatic choice.

4.2.3 Controller

The controller package is responsible for the communication between the view and the model. It takes requests from the view and delegates to the logic of the model package. These are compact Python classes, but they are especially important. Model classes on their own are just isolated blocks of business logic, and the view is just asking information through the API without the knowledge of the existence or structure of model. The controllers are what tie Authenstickator together.

The entry point of everything is the `MasterController`. This is why, on Figure 4.2, every communication path from package script goes directly to it. The master controller is passed as the `js_api` parameter of pywebview's `create_window` method. This makes it so that JavaScript can call methods of `MasterController`. Then, following the same pattern as for HTML and JS files, each view has its own controller: `RegisterController`, `LoginController`, `TOTPController`, and `ChangePasswordController`. These are defined as fields of `MasterController`, this way JS can call them as shown in Table 4.1.

Call in JavaScript	**Method in Python**
window.pywebview.api.function()	MasterController.function()
window.pywebview.api.register.function()	RegisterController.function()
window.pywebview.api.login.function()	LoginController.function()
window.pywebview.api.totp.function()	TOTPController.function()
window.pywebview.api.change_password.function()	ChangePasswordController.function()

Table 4.1: How API calls map to controller functions.

The fact that the fields of `MasterContoller` were automatically picked up by `pywebview` is really convenient, but it also came with a surprise. When testing on Windows, the app crashed unexpectedly. This was because the `MasterController` class has a field

`window`, which holds reference to the return value of `create_window`, i.e., the window of the application. Since it is a field, pywebview tried to serialize it to make it available for JS to call. For some reason, this was not a problem on Linux, but it was a problem on Windows. The solution was to prepend the name of the class field with a dunder (`window` *rightarrow* `_window`). This is a common convention in Python to mark private fields; pywebview than knows that I do not wish to expose to JS this field.

Communication: an example and the conventions.

The easiest way of grasping how end-to-end communication works is to go through an example. In the following, I present you a full cycle of what happens when the user clicks Enter on the register view (see: Figure 4.4a). The flow is somewhat simplified, in code other things happen as well, but for the sake of showing a communication example, I included only the steps relevant for this purpose.

1. The user clicks `newPasswordButton` (of `register.html`).
2. The click of `newPasswordButton` triggers the call of function `newPasswordHandler` (of `registerScript.js`).
3. `newPasswordHandler`, through function `callApi` (of `ScriptUtils.js`), calls `window.pywebview.api.register.new_password_handler`.
4. Function `new_password_handler` (of `register_controller.py`) picks up the API call.
5. `new_password_handler` calls function `set_password` (of `password.py`).
6. `set_password` sets the password. Returns `True` on success, `False` on failure.
7. Based on the response from `set_password`, `new_password_handler` returns a `Response` object of `SuccessType` on success, `ErrorType` on failure.
8. `newPasswordHandler`, through `callApi`, receives the `Response` object and shows to the user the success message on success, the error message on failure.

First, you can see that there is a certain naming convention used, which helps a lot in finding related methods. The element, the JS handler, and the Python handler all use the same name, but with different suffixes and conventions. The HTML element contains its type (button, label, input). The JS handler follows the camelCase convention of JS, and adds the suffix "handler". The Python handler had the same name as the JS handler, but following the underscore naming convention of Python.

Then, there are the data type conventions. The classes of model do not follow a strict guideline: some of them return `True`/ `False` on success/ error, some of them return `None` of failure, and some of them needed no error handling, so they had no failure scenario. It all depends on the purpose of the function and the context. However, the communication between the view and the controller follows a strict pattern. Each Python handler shall return `ResponseType`, defined in `response.py` as `SuccessType | ErrorType`. `SuccessType` is a dict which has status attribute equal to `STATUS_SUCCESS`, a success message, and optionally a data field, used to send the return value of the model classes if it's the case. `ErrorType` has status `STATUS_ERROR`, and an error message.

The two constants, `STATUS_SUCCESS` and `STATUS_ERROR`, are shared with the frontend at startup. Hence, when JS handlers receive a response from the Python handlers, they can deduce if a response is of success or error, and can further extract fields based on this information.

4.3 Development

Authenstickator was developed on a relatively old laptop running Pop!_OS 22.04 LTS. For source control, just like every sane developer, I used Git. The project is available as a public GitHub repository at `https://github.com/broland29/authenstickator`. It has GPL-3.0 license, meaning it can be used freely as long as the product which uses is also open source.

The IDE used for development was PyCharm 2026.1.4. The default Python formatter was used (Settings → Editor → Code Style → Scheme: Default), with a hard wrap at 100 columns. 100 characters is reasonable for Python code, for JS, HTML and CSS I did not set such limits. I also set to format the code on each save (Settings → Actions on Save → tick Reformat code and Optimize imports).

The documentation was written in Latex, using Visual Studio Code 1.128.0 and the LaTeX Workshop extension. For spell checking, the LTeX+ extension was used, which is completely offline. The source code of the documentation is also under version control, available at `https://github.com/broland29/authenstickator-docs`, under the same license. The starting point of the documentation was the template provided by the Technical University of Cluj-Napoca, available at `https://cs.utcluj.ro/studii-de-masterat/sesiunea-ii-septembrie-2026.html`.

4.3.1 Dependency Management, Imports

For installing packages and keeping track of their versions, `pip` was used. Some packages installed through `pip` also have OS-related dependencies, which will be discussed in later sections. But, in principle, `pip` was the one and only source of packages. Then, a `requirements.txt` file was generated using pip freeze.

Listing 4.4: Generating the requirements file.

```
pip freeze > requirements.txt
```

This file contains all the packages and their versions. Even though Authenstickator was developed with cross-platform portability in mind, some functionalities, like the usage of TPM, required platform-specific packages. These are also marked in the requirements file, like: `sys_platform == "linux"`. If, during development, new dependencies are added, or versions are updated, the command 4.4 has to be run again. But, be careful with updating packages: while I tried bringing everything to the latest version, there are some packages which simply do not work with newer versions, like `PyGObject`.

Dependencies are not installed directly on the whole system, but a virtual environment is used. A virtual environment (`venv`) is used to isolate Python packages used for Authenstickator from other Python packages on the system. This is standard dependency management, and ensures not just that Authenstickator works well, but that we don't break other apps. Each developer creates its own virtual environment and these are not pushed to git (specified in `.gitignore`).

The application is run as a module (with the `-m` flag). It is important to have every import start from the `src` folder. Even if relative imports work in PyCharm (due to hacks PyCharm does in the background), not following this convention will have nasty side effects: running from the CLI with the usual command will simply not work. The same convention has to be kept in tests also, since I hit unwanted side effects due to importing files differently in tests than in source code. Make sure to set `authenstickator` as a Source Root (see: [9]).

4.3.2 Setup and Packing

Authenstickator is meant to run on a USB Stick, on machines which possibly do not even have Python. One such release candidate is made with the help of PyInstaller. "PyInstaller bundles a Python application and all its dependencies into a single package. The user can run the packaged app without installing a Python interpreter or any modules. (...) However, it is not a cross-compiler; to make a Windows app you run PyInstaller on Windows, and to make a Linux app you run it on Linux, etc. "[10]

Packing commands also differ on Linux and on Windows. For example, executables' icons are treated differently. For Linux, a lot of desktop environments (GNOME, KDE, Xfce) use `.desktop` files to tie the icon to the application launcher. Since these files need absolute paths, and the app is run on a USB stick, I simply gave up on giving an icon for the Linux version. Power users are welcome to create their own `.desktop` files. The icon is available in the source code as `logo.png`. For Windows, a `.ico` file is required, which can be specified as an argument for the PyInstaller, and will be embedded inside the `exe` file.

4.3.3 Setup on Linux

The following setup was tested on Ubuntu 22.04 LTS. It might work on other Debian-based systems as well, but there is no guarantee. Development was done with Python 3.10.12, which came with the OS installation.

Some Python dependencies from `requirements.txt` require specific Linux packages to work. These include: a build compiler, the TPM2 development headers and build tools, some FAPI requirements (TPM-related), and some `PyGObject` requirements (`pywebview`/ UI related). For development, to avoid TPM lockout, a software TPM, `swtpm` is recommended, which also needs installation. Be careful with the `apt install` command: if you accidentally leave spaces after the line-breaking backslashes, it won't work. After it finishes, the project is cloned, a virtual environment is created, and requirements are installed inside.

Listing 4.5: Setup commands for Linux.

```
sudo apt-get update

sudo apt install -y git \
  python3.10 python3.10-venv \
  build-essential \
  libtss2-dev pkg-config python3-dev \
```

[9] https://stackoverflow.com/questions/52949551/is-there-any-way-to-force-pycharm-to-always-use-absolute-imports

[10] [**?**]

```
  libssl-dev libjson-c-dev libcurl4-openssl-dev \
  libgirepository1.0-dev libcairo2-dev libglib2.0-dev gobject-
     introspection \
  swtpm

git clone https://github.com/broland29/authenstickator.git
cd authenstickator

python3 -m venv venv
source venv/bin/activate

pip install -r requirements.txt
```

Once the scripts from 4.5 all finished, you can start the app. Always activate the virtual environment before running.

Listing 4.6: Running the app on Linux.

```
python3 -m src.main
```

To ensure that the app is properly working, you can run the unit tests:

Listing 4.7: Running the tests on Linux.

```
python -m pytest
```

4.3.4 Provisioning the TPM on Linux

If you want to use Authenstickator with the TPM feature enabled, you need to provision the TPM. If not, you can skip this section.

Real TPM

Check if you actually have a TPM chip.

Listing 4.8: Check if you have a TPM.

```
sudo dmesg | grep -i tpm
ls -l /dev/tpm0 /dev/tpmrm0
```

Having a physical TPM chip (or a simulated one, if you are on a VM) is a hard requirement for TPM functionalities. See Figure 4.7 for example outputs if you have it or not.

For provisioning, you will need to run the `tss2_provision` command of package `tpm2-tools`. You can try getting it through `apt`, but it did not work for me.

```
sudo apt install -y tpm2-tools
sudo -u tss tss2_provision
```

If you get a command not found error, welcome to the club. The alternative is to compile `tpm2-tools` from source. This package requires the `tpm2-tss` package, which also needs to be compiled from source with a compatible version. I generated a script with Gemini, found in the repo under `script/setup_fapi.sh`. It can be run as shown in Listing 4.9. This script:

Figure 4.7: Check if you have a TPM.

1. Installs dependencies for `tpm2-tss` and `tpm2-tools`.
2. Adds your user to the `tss` group. You will run Authenstickator as your user, and interacting with TPM requires membership of this group.
3. Compiles `tpm2-tss v4.1.3`.
4. Compiles `tpm2-tools v5.6`.
5. Removes some FAPI profiles which were too modern for Ubuntu 22.04 and adds `ek_cert_less = "yes"` to the automatically generated `fapi-config.json`.
6. Creates the directories necessary for provisioning, and provisions.
7. Generates a random number to see if TPM responds (although this does not require provisioning).

Listing 4.9: Running the FAPI setup script.

```
chmod +x ./script/setup_fapi.sh
sudo ./script/setup_fapi.sh
```

If you mess with the TPM too much, it will lock you out. TPM is protecting itself against dictionary attacks (DA), by disallowing further commands for a period of time. This time period can be pretty long (don't ask me why I have this knowledge). One possible solution to get out of the lockout is a command I found in a thread from Ubuntu[11], shown in Listing 4.10. The file it modifies contained the value 0 when I was locked out, so it is probably a reversed violation counter. Without a system reboot, the change has no effect.

Listing 4.10: Getting out of TPM's Lockout Mode.

```
echo 5 | sudo tee /sys/class/tpm/tpm0/ppi/request
reboot
```

Also, if you get provisioning errors saying that TPM was already set up, you might want to try deleting previous keystores (see: Listing 4.11). These affect only FAPI, not the whole TPM (I have disk encryption which uses TPM, but this command did not

[11] `https://documentation.ubuntu.com/core/how-to-guides/manage-ubuntu-core/troubleshooting/`

break it). If other software on your PC uses FAPI, you might want to look for a more graceful solution.

Listing 4.11: Wiping previous keystores.

```
sudo rm -r /var/lib/tpm2-tss/system/keystore/
rm -r ~/.local/share/tpm2-tss/user/keystore/
```

The default `fapi_config` is generated while compiling tpm2-tss and tpm2-tools. It can be found at the path `/etc/tpm2-tss/fapi-config.json`. After the script's modifications, it might look like on Listing 4.12.

Listing 4.12: The default `fapi-config.json`, after the script's modifications.

```
{
  "profile_name": "P_ECCP256SHA256",
  "profile_dir": "/etc/tpm2-tss/fapi-profiles/",
  "user_dir": "~/.local/share/tpm2-tss/user/keystore",
  "system_dir": "/var/lib/tpm2-tss/system/keystore",
  "tcti": "",
  "system_pcrs": [],
  "log_dir": "/var/run/tpm2-tss/eventlog/",
  "firmware_log_file": "/sys/kernel/security/tpm0/
   binary_bios_measurements",
  "ima_log_file": "/sys/kernel/security/ima/binary_runtime_measurements
   ",
  "ek_cert_less": "yes"
}
```

Virtual TPM

To run with a virtual TPM, change the configs to enable it (see: Section 4.2.1). It should work out of the box. But if you have to troubleshoot, you can check manually if `swtpm` is running using `netcat` (see: Listing 4.13). While Authenstickator runs, the command should output something like "Connection to localhost (127.0.0.1) 2322 port [tcp/*] succeeded!". Run it multiple times, the first one might fail. Make sure the ports match the ones in configs.

Listing 4.13: Checking if the virtual TPM is running.

```
nc -zv localhost 2322
```

If `swtpm` locks you out and/or you want to start fresh, you can kill its process and delete its folder.

```
pkill swtpm | rm -r /tmp/vtpm
```

Then, you can re-launch the app, or run the commands from `linux_tpm.py` manually.

4.3.5 Packing on Linux

To pack Authenstickator, after a successful setup (see: Section 4.3.3), run the Linux packing script, as in Listing 4.14. If you ran it previously, it will prompt you if you want to overwrite existing data, and you need to press Y. The flags specified in `pack-linux.sh` add a name to the packed executable, add non-Python files which `pyinstaller` would

not include by default, ensure paths start from root, and specify the output directory. Builds are generated in the temporary folder, so they will be wiped at reboot; you can change this if you wish.

Listing 4.14: Running the packing script for Linux.

```
source venv/bin/activate
pip install pyinstaller
./script/pack-linux.sh
```

If you want to create a release/archive, navigate to the previous command's output (by default `/tmp/authenstickator/dist`) and pack it using `tar`. See Listing 4.15 for an example of packing for an Ubuntu release, and later extracting. If you did this on a VM and want to get the archive to your host machine, on the VM, you can get your IP (`YOUR-VM-IP`) using `ip a`, then start a local HTTP server with Python using `python3 -m http.server 8000`. After that, you can connect through your host's browser through `http://YOUR-VM-IP:8000/`.

Listing 4.15: Create archive and extract on Linux.

```
tar -czvf Authenstickator-v1.0.0-Ubuntu-22.04.5-LTS.tar.gz
   Authenstickator/
tar -xzvf Authenstickator-v1.0.0-Ubuntu-22.04.5-LTS.tar.gz
```

4.3.6 Setup on Windows

The following setup was tested on Microsoft Windows Version 10.0.19045.6466. It might work on other Windows systems as well, but there is no guarantee.

Although development was done with Python 3.10.12, for Windows, this version does not have a binary release. Instead, you can get 3.10.11 from `https://www.python.org/downloads/release/python-31011/`. When running the installer, tick the option to add to path and to disable path length limit. You will also need Git, you can get it from `https://git-scm.com/install/windows`. After having Python and Git, run the commands from Listing 4.16. Command `where python` shall output as first entry `\venv\Scripts\python.exe`; if not, try activating again/ in a new terminal (it happened to me).

Listing 4.16: Setup commands for Windows.

```
git clone https://github.com/broland29/authenstickator.git
cd authenstickator
python -m venv venv
venv\Scripts\activate.bat
where python
pip install -r requirements.txt
```

If you get "ImportError: DLL load failed while importing zxingcpp: The specified module could not be found.", get `vc_redist.x64.exe` from `https://aka.ms/vs/17/release/vc_redist.x64.exe`.

Once the scripts from 4.16 all finished, you can start the app. Always activate the virtual environment before running.

Listing 4.17: Running the app on Linux.

```
python -m src.main
```

To ensure that the app is properly working, you can run the unit tests:

Listing 4.18: Running the tests on Linux.

```
python -m pytest
```

Packing on Windows

To pack Authenstickator, after a successful setup (see: Section 4.3.6), run the Windows packing script, as in Listing 4.19. If you ran it previously, it will prompt you if you want to overwrite existing data, and you need to press Y. The flags specified in `pack-windows.bat` add a name and icon to the packed executable, add non-Python files which `pyinstaller` would not include by default, ensure paths start from root, disable the scary black terminal, and specify the output directory. Builds are generated in the temporary folder; you can change this if you wish.

Listing 4.19: Running the packing script for Windows.

```
venv\Scripts\activate.bat
pip install pyinstaller
script\pack-windows.bat
```

If you want to create a release/archive, navigate to the previous command's output (by default `C:\Users\YOUR-USERNAME\AppData\Local\Temp\authenstickator\dist`) and pack it using `tar`. See Listing 4.15 for an example of packing for an Ubuntu release, and later extracting. If you did this on a VM and want to get the archive to your host machine, on the VM, you can get your IP (`YOUR-VM-IP`) using `ipconfig`, then start a local HTTP server with Python using `python -m http.server 8000` . After that, you can connect through your host's browser through `http://YOUR-VM-IP:8000/`.

Listing 4.20: Create archive and extract on Windows.

```
tar -a -c -f Authenstickator-v1.0.0-Windows-10.zip
   Authenstickator
tar -x -f Authenstickator-v1.0.0-Windows-10.zip
```

As the output says, the result is in your temporary folder, by default `C:`
`Users`
`YOURUSERNAME`
`AppData`
`Local`
`Temp`
`authenstickator`
`dist`.

4.4 User Manual

4.4.1 USB Setup on Linux

4.4.2 USB Setup on Windows

4.4.3 App functionalities

Chapter 5

Theoretical and Experimental Results

5.1 Unit Testing

5.2 Manual Testing

5.3 Testing on Linux VM

Since the most popular distro is Ubuntu[1], I chose to test and build on a Ubuntu 22.04 LTS machine. Ubuntu 20.04 LTS hit its Standard Security Maintenance deadline at May 2025, while 22.04 LTS is still in free support until May 2027. Distros shall be backward compatible, so the results shall be the same for newer Ubuntu releases as well, and the setup will probably work on Debian and other Debian-based distros as well.

If you wish to test the application with the real TPM setting, but still want to avoid using your own TPM and getting locked out, you can use a Virtual Machine. My recommendation is QEMU/KVM, since it is easy to set up and works well. The following steps for TPM setup are for QEMU/KVM 4.0.0. Turn of the virtual machine if it is currently running. Inside the "Show virtual hardware details" panel (the icon containing the letter I, when the VM is selected), click "Add Hardware". Select TPM. Inside "Advanced options", select Model "CRB" and Version "2.0". Click finish, and start the VM.

5.4 Testing on Windows VM

For unit tests, the `pytest` library was used. To run all tests, with the virtual environment activated, run the following command:

```
python -m pytest
```

[1] 2025 Stack Overflow Developer Survey: `https://survey.stackoverflow.co/2025/technology`

Chapter 6

Conclusions

6.1 Further improvements

Authenstickator does not support TPM operations on Windows. While I would have loved to make it work on Windows as well, it simply looks like TPM development is not meant to be done on Windows unless you are a TPM expert. The `tpm2-pytss` library does not support Windows[1], and there are no similar libraries for Windows. So writing equivalent FAPI code was not an option. The remaining solution was to go low level, calling the TPM 2.0 commands directly (`TPM2_NV_ReadPublic`, `TPM2_GetRandom`, `TPM2_NV_DefineSpace`, `TPM2_NV_Write`, `TPM2_NV_Read`). The only way I found to call these was through the `Tbsip_Submit_Command` C API.[2] This API expects you to build the commands byte by byte. I found no examples online of how to use it. AI sketched me an example of how the code working on Linux would look like with calls to this API, and it gave me more than 200 lines of byte-manipulating code. Unfortunately, my current TPM knowledge is not deep enough to write and document such code, nor do I wish to write such low-level code for something that should be a few lines, but isn't, since Windows. I also had the idea to use WSL, but it looks like WSL has no access to the physical TPM.

[1] `https://github.com/tpm2-software/tpm2-pytss/issues/597#issuecomment-2556916125`

[2] `https://learn.microsoft.com/en-us/windows/win32/api/tbs/nf-tbs-tbsip_submit_command`

Bibliography

[1] yubico, "How the yubikey works," https://www.yubico.com/products/how-the-yubikey-works/, accessed: August 21, 2026.

[2] T. Roche, "Eucleak side-channel attack on the yubikey 5 series (revealing and breaking infineon ecdsa implementation on the way)," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 4026–4043, also available online: https://ninjalab.io/wp-content/uploads/2024/09/20240903_eucleak.pdf.

[3] Fortinet, "What is two-factor authentication (2fa), and how can it be enabled?" https://www.fortinet.com/resources/cyberglossary/two-factor-authentication, accessed: August 8, 2026.

[4] B. Schneier, "Two-factor authentication: too little, too late," *Communications of the ACM*, vol. 48, no. 4, p. 136, 2005.

[5] L. Meyer, S. Romero, G. Bertoli, T. Burt, A. Weinert, and J. Lavista Ferres, "How effective is multifactor authentication at deterring cyberattacks?" 05 2023.

[6] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, "HOTP: An HMAC-Based One-Time Password Algorithm," RFC 4226, Dec. 2005. [Online]. Available: https://www.rfc-editor.org/info/rfc4226/

[7] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, May 2011. [Online]. Available: https://www.rfc-editor.org/info/rfc6238/

[8] National Institute of Standards and Technology (NIST), "Advanced encryption standard (aes)," U.S. Department of Commerce, Federal Information Processing Standards Publication (FIPS) 197-upd1, May 2023. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf

[9] F. Estudio, "Rijndael (AES) animation," 2007, originally made in 2004, enhanced for the CrypTool project. [Online]. Available: https://formaestudio.com/rijndaelinspector/archivos/Rijndael_Animation_v4_eng-html5.html

[10] B. Kaliski, "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, Sep. 2000. [Online]. Available: https://www.rfc-editor.org/info/rfc2898/

[11] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: The memory-hard function for password hashing and other applications," March 2017, version 1.3 of Argon2: PHC release. [Online]. Available: https://github.com/P-H-C/phc-winner-argon2/blob/master/argon2-specs.pdf

[12] D. Anton and E. Simion, “Linux unified key setup (luks) - the good, the bad, the ugly,” 04 2019.

[13] W. Arthur, D. Challener, and K. Goldman, *A Practical Guide to TPM 2.0*. Apress, 2015.

[14] A.-L. Marx, “Building trust - use cases and implementation of TPM 2.0 in embedded Linux systems,” Presentation slides at Embedded Recipes 2025, Nice, France, May 2025. [Online]. Available: https://embedded-recipes.org/2025/images/slides/2025_EmbeddedRecipes_BuildingTrust.pdf

[15] pi3g, “From application to tpm - linux deepdive,” https://www.youtube.com/watch?v=PF5Uv16kJ-M, October 2025, accessed: August 8, 2026.